

Membrane Gas Separation Simulation with OpenFOAM: A Short Practical Course

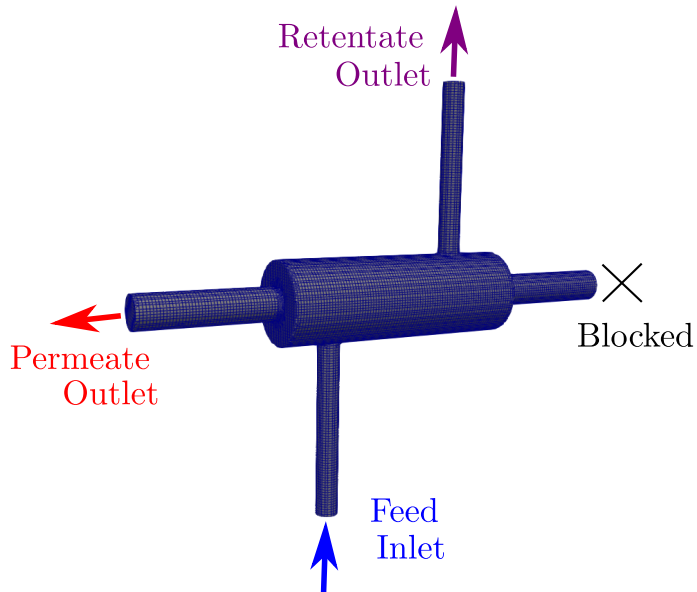
Lecture 02

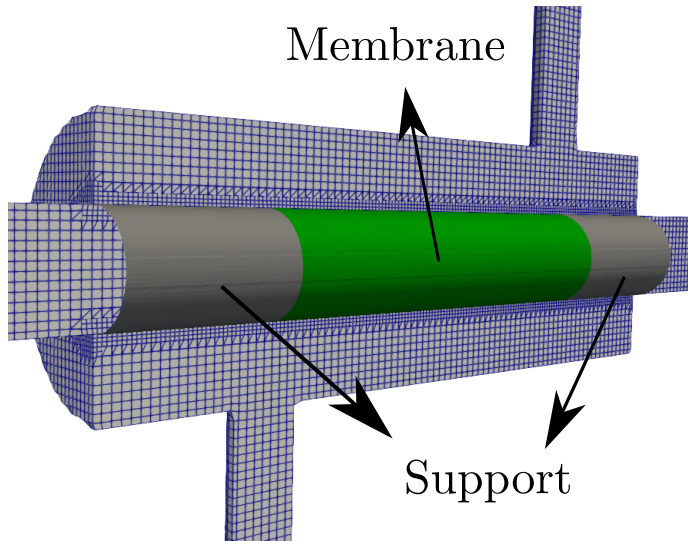
Professor: Adriano Possebon Rosa (aprosa@unb.br)

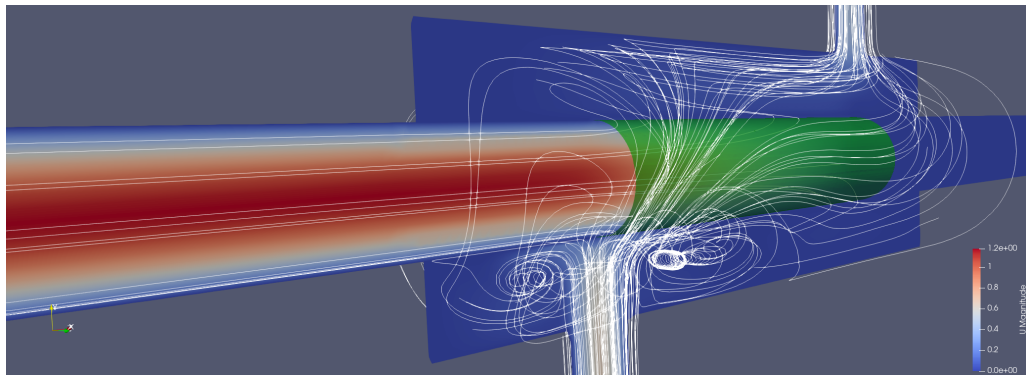
Energy and Environment Laboratory (lea.unb.br)

University de Brasília

Goal: simulate a membrane gas separation module using Computational Fluid Dynamics (CFD).







Lectures:

- **Lecture 00:** Membrane Gas Separation; Governing Equations; CFD Basics; Finite Volume Method; OpenFOAM; Simulation: Lid-Driven Cavity.
- **Lecture 01:** Governing Equations (Basic Problems); Simulation: Parallel Plates; Mesh Basics; Mesh Generation with Gmsh; Simulation: T-Junction; Baffles.
- **Lecture 02:** Gas Flow Simulation; Baffles (Membranes); Simulation: T-Junction with Membrane; SnappyHexMesh; Simulation: 3D T-Junction with Membrane and SHM.
- **Lecture 03:** Running the Complete Simulation (Membrane Module); Simulation Setup; Boundary Conditions; Initial Conditions; Simulation and Experiment; Parallelization; Post Process.

In this lecture, we will cover the final components needed to complete the membrane gas separation module simulation.

We are already familiar with the fundamentals of **OpenFOAM**. Now, we will learn how to:

- Simulate the flow of a **multicomponent gas mixture**;
- Apply the `baffle` utility to create a **permeable membrane**;
- Use `snappyHexMesh` to generate a **3D mesh** from a given geometry.

We will base our study on **six simulations**.

Each simulation introduces one new detail or concept.

All simulations are available in the *simulations* folder.

Each simulation folder contains a `scriptRun.py` file (a Python script that automates the entire simulation process). You can use this script to understand the sequence of steps required to run the case.

Sumário

- 1 Gas Flow Simulation
- 2 Baffles (Membranes)
- 3 Simulation: T-Junction with Membrane
- 4 SnappyHexMesh
- 5 Mesh Generation with SHM
- 6 Simulation: 3D T-Junction with Membrane and SHM
- 7 Conclusion

Macroscopic Modeling - **Governing Equations**

Mass conservation equation:

$$\frac{\partial}{\partial t} (\rho) + \nabla \cdot (\rho \mathbf{U}) = 0$$

Momentum conservation equation:

$$\frac{\partial}{\partial t} (\rho \mathbf{U}) + \nabla \cdot (\rho \mathbf{U} \mathbf{U}) = -\nabla p + \nabla \cdot \left(\mu \left(\nabla \mathbf{U} + (\nabla \mathbf{U})^T \right) \right)$$

Energy conservation equation:

$$\frac{\partial}{\partial t} (\rho c_p T) + \nabla \cdot (\rho c_p T \mathbf{U}) = \nabla \cdot (k \nabla T)$$

Mass concentration equation:

$$\frac{\partial}{\partial t} (\rho Y_i) + \nabla \cdot (\rho Y_i \mathbf{U}) = \nabla \cdot (\rho D_{m,i} \nabla Y_i) + S_{m,i}$$

$$S_{m,i} = \frac{c_i A}{V_c} (p_{i,R} - p_{i,P})$$

Y_i : mass fraction of species i ; $D_{m,i}$: mass diffusion coefficient of species i ;
 $S_{m,i}$: mass transfer source term for species i .

The gas is modeled as an ideal gas:

$$pV = nR_uT$$

The viscosity is calculated using Sutherland's model:

$$\mu = \frac{A_s \sqrt{T}}{1 + T_s/T}$$

Thermal conductivity:

$$k = \mu c_v (1.32 + 1.77R/c_v)$$

The governing equations follow a standard pattern.

We can generalize these equations for any transported fluid property ϕ :

$$\frac{\partial (\rho\phi)}{\partial t} + \nabla \cdot (\rho \mathbf{U} \phi) = \nabla \cdot (\Gamma_\phi \nabla \phi) + S_\phi$$

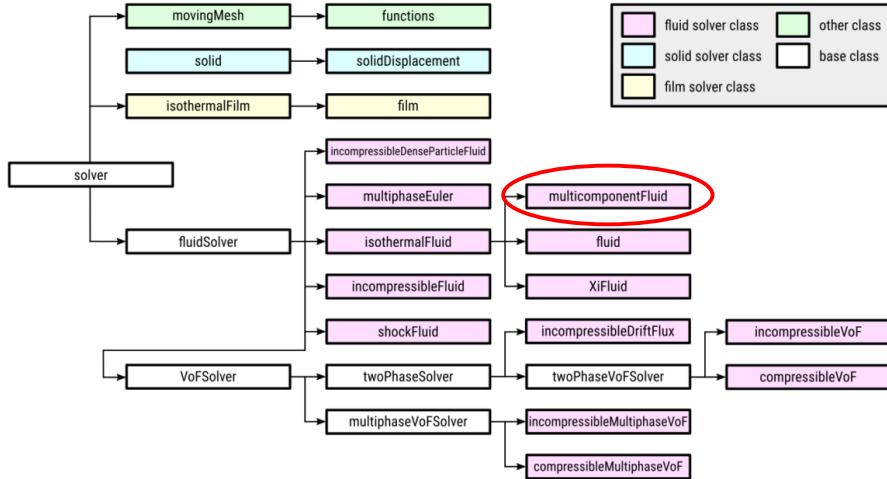
$\frac{\partial(\rho\phi)}{\partial t}$ is the transient term, representing the rate of change of ϕ within the control volume.

$\nabla \cdot (\rho \mathbf{U} \phi)$ is the advective flux term, representing the transport of ϕ due to fluid flow.

$\nabla \cdot (\Gamma_\phi \nabla \phi)$ is the diffusive transport term, representing the transport of ϕ by diffusion.

S_ϕ is the source term, representing the rate of generation or destruction of ϕ , per unit volume.

We are going to use the OpenFOAM module *multicomponentFluid*.



multicomponentFluid.H file:

```
1 Class
2     Foam::solvers::multicomponentFluid
3
4 Description
5     Solver module for steady or transient turbulent flow of compressible
6     multicomponent fluids with optional mesh motion and change.
7
8     Uses the flexible PIMPLE (PISO-SIMPLE) solution for time-resolved and
9     pseudo-transient and steady simulations.
10
11     Optional fvModels and fvConstraints are provided to enhance the simulation
12     in many ways including adding various sources, chemical reactions,
13     combustion, Lagrangian particles, radiation, surface film etc. and
14     constraining or limiting the solution.
15
16 Reference:
17 \verbatim
18     Greenshields, C. J., & Weller, H. G. (2022).
19     Notes on Computational Fluid Dynamics: General Principles.
20     CFD Direct Ltd.: Reading, UK.
21 \endverbatim
```

thermophysicalPredictor.C file (solves the mass fraction Y_i equation):

```
1  forAll(Y, i)
2  {
3      if (composition.solve(i))
4      {
5          volScalarField& Yi = Y_[i];
6          fvScalarMatrix YiEqn
7          (
8              fvm::ddt(rho, Yi)
9              + mvConvection->fvmDiv(phi, Yi)
10             + thermophysicalTransport->divj(Yi)
11             ==
12             reaction->R(Yi)
13             + fvModels().source(rho, Yi)
14         );
15
16         YiEqn.relax();
17         fvConstraints().constrain(YiEqn);
18         YiEqn.solve("Yi");
19         fvConstraints().constrain(Yi);
20     }
21 }
```

The OpenFOAM tutorial that we are going to use as a base case is the membrane tutorial:

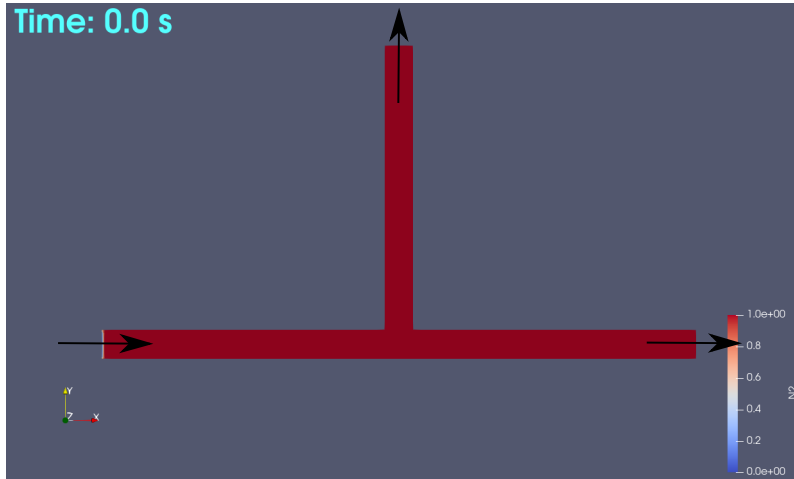
Tutorial: *openfoam11/tutorials/multicomponentfluid/membrane*

Let's start the simulations.

In all simulations of this lecture, we will consider an initial condition where the domain is initially **filled with** N_2 . The inlet gas enters the domain with a composition of **50% CO_2** and **50% CH_4** , specified in terms of **mass fraction**.

Important note: OpenFOAM uses **mass fraction**. In this lecture, we will use mass fraction. In the next lecture, we are going to use molar fraction, as it is the most common species concentration used in the literature.

Simulation 00: Gas in a 2D T-Junction.



From Lecture 01, we already know how to create a T-junction geometry using `gms`.

We also learned how to configure several key files in the `0`, `constant`, and `system` folders.

In this case, we will use the `multicomponentFluid` solver. This requires a few additional configuration steps to properly set up the simulation.

We need to inform the gas properties in the physical properties file.

constant/physicalProperties file:

```
1 thermoType
2 {
3     type            hePsiThermo;           // Thermodynamic model type
4     mixture          multicomponentMixture; // Mixture handling
5     transport         sutherland;           // Transport properties model
6     thermo            janaf;                // Thermodynamic coefficients
7     energy            sensibleEnthalpy;     // Energy formulation
8     equationOfState   perfectGas;           // EoS model
9     specie            specie;               // Specie definition
10 }
11
12 defaultSpecie CH4;
13
14 #include "physicalPropertiesValues"
```

type: hePsiThermo

- Most common thermodynamic package for compressible flows
- Uses psi (compressibility) formulation

mixture: multicomponentMixture

- Handles gas mixtures (e.g., combustion)
- Alternative: pureMixture for single-component flows

transport: sutherland

- Temperature-dependent viscosity model
- Good for high-temperature flows

thermo: janaf

- Polynomial coefficients for $c_p(T)$, $h(T)$ from JANAF tables

energy: sensibleEnthalpy

- Tracks thermal energy component only (relative to reference)

equationOfState: perfectGas

- Ideal gas law ($p = \rho RT$)
- Most common for gas dynamics

specie: specie

- Basic species definition template
- Contains molecular weight, composition data

Note: These selections are typical for gas flows

We need to specify which species are simulated.

constant/physicalPropertiesValues file:

```
1 species
2 (
3     CH4
4     CO2
5     N2
6 );
```

The first one in the list is the *default specie*.

Then we need to define the properties of each specie individually.

constant/physicalPropertiesValues file:

```
7  CH4
8  {
9      specie
10     {
11         molWeight      16.0428;
12     }
13     thermodynamics
14     {
15         Tlow           200;
16         Thigh          6000;
17         Tcommon        1000;
18         highCpCoeffs   ( 1.63543 0.0100844
19                         -3.36924e-06 5.34973e-10 -3.15528e-14 -10005.6 9.9937 );
20         lowCpCoeffs    ( 5.14988 -0.013671
21                         4.91801e-05 -4.84744e-08 1.66694e-11 -10246.6 -4.64132 );
22     }
23     transport
24     {
25         As             1.0194e-06;
26         Ts             171.06;
27     }
28 }
```

For each species, we need to include a corresponding file in the *0* folder.

In all simulations of this lecture, we will consider an initial condition where the domain is initially **filled with N₂**. The inlet gas enters the domain with a composition of **50% CO₂** and **50% CH₄**, specified in terms of **mass fraction**.

Therefore, we must include the following files in the *0* folder:

- Species mass fraction fields: CH₄, CO₂, and N₂;
- Flow and thermodynamic fields: U, p, and T.

```
1 dimensions      [0 0 0 0 0 0 0];
2
3 internalField uniform 0.0;
4
5 boundaryField
6 {
7
8     inlet
9     {
10         type      fixedValue;
11         value      uniform 0.5;
12     }
13
14     ...
```

Note the internalField (initial condition) and the inlet value (mass concentration of CH4 in the inlet gas).

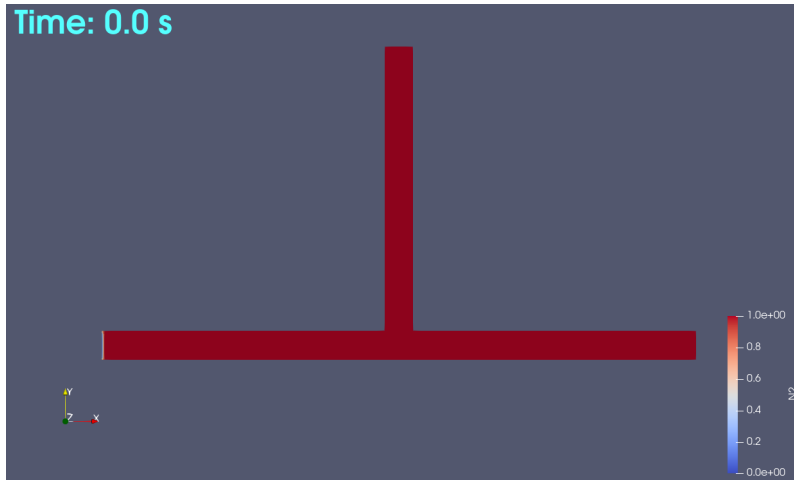
0/CO2

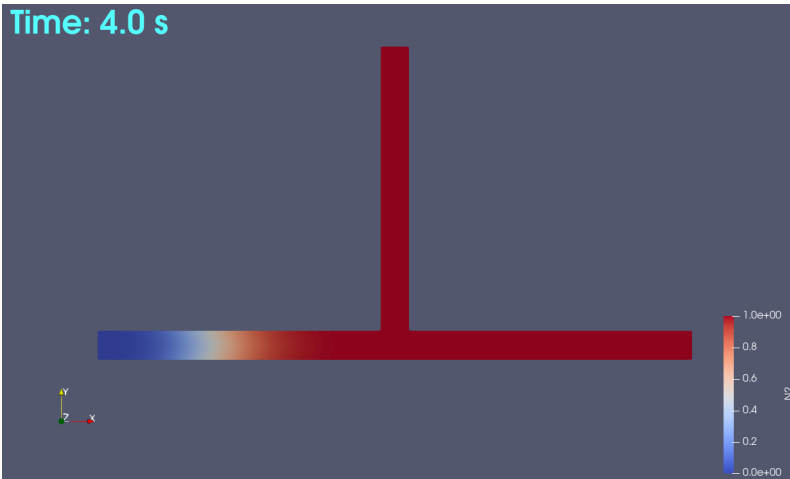
```
1 dimensions      [0 0 0 0 0 0 0];
2
3 internalField uniform 0.0;
4
5 boundaryField
6 {
7
8     inlet
9     {
10         type      fixedValue;
11         value      uniform 0.5;
12     }
13
14     ...
```

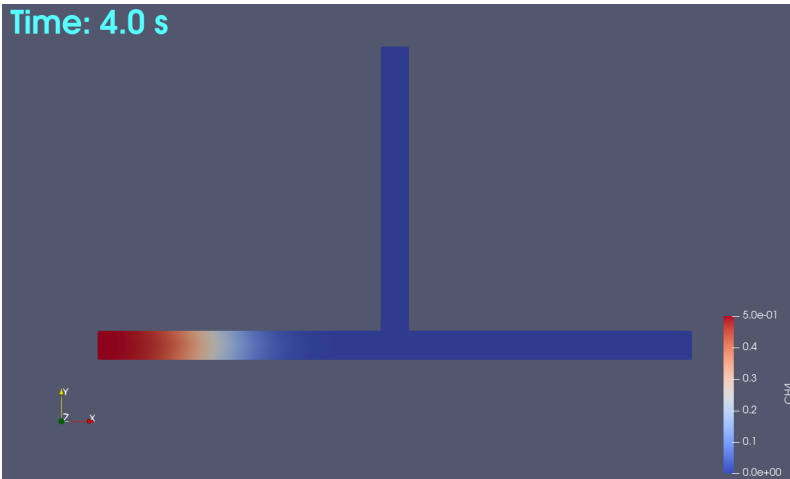
0/N2

```
1 dimensions      [0 0 0 0 0 0 0];
2
3 internalField uniform 1.0;
4
5 boundaryField
6 {
7
8     inlet
9     {
10         type      fixedValue;
11         value      uniform 0.0;
12     }
13
14     ...
```

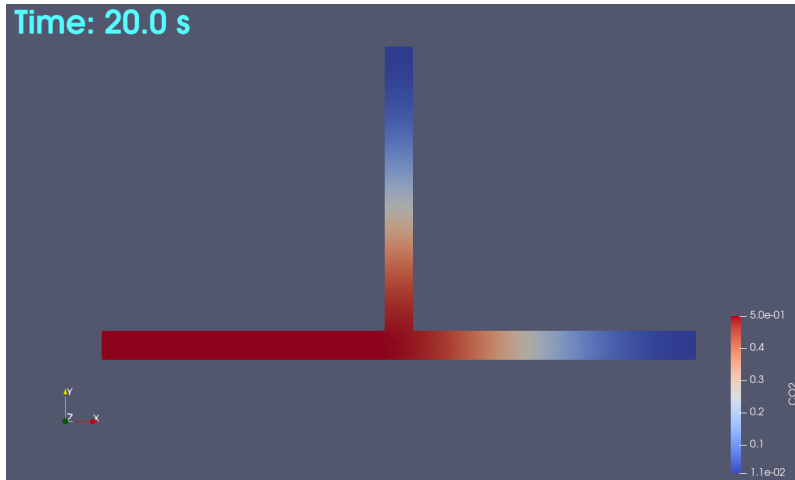
Initial condition. N2 concentration of 1 in the entire domain.



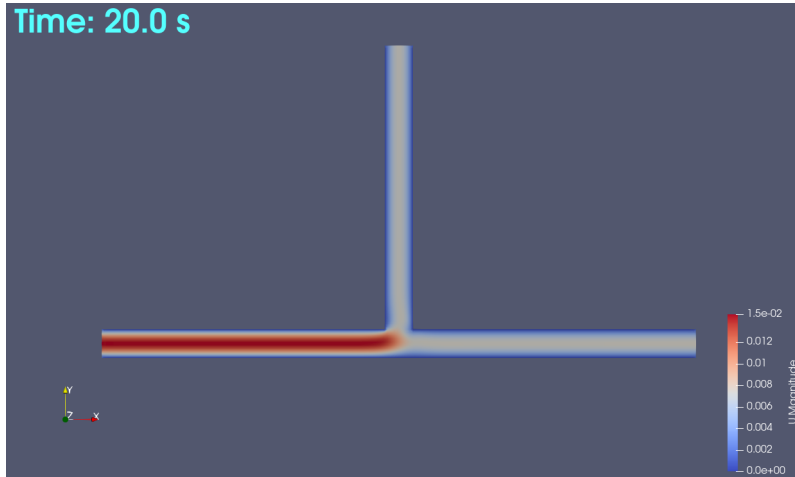


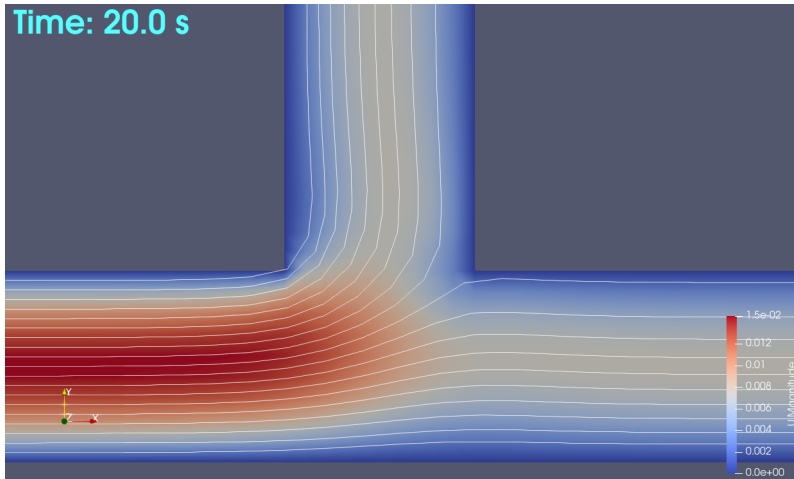


CO_2 concentration.



Velocity magnitude.

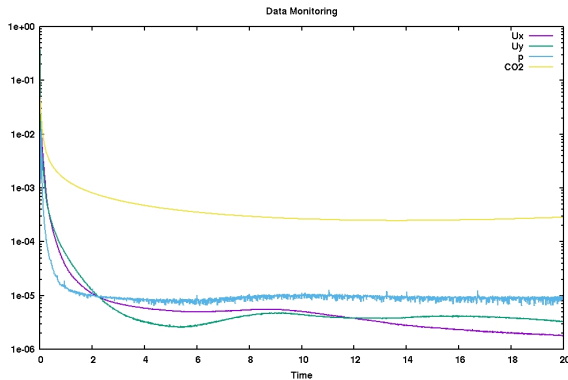




You can execute the command

```
$ foamMonitor -l postProcessing/residuals/0/residuals.dat
```

to visualize the residuals of the simulation.



Sumário

- 1 Gas Flow Simulation
- 2 Baffles (Membranes)
- 3 Simulation: T-Junction with Membrane
- 4 SnappyHexMesh
- 5 Mesh Generation with SHM
- 6 Simulation: 3D T-Junction with Membrane and SHM
- 7 Conclusion

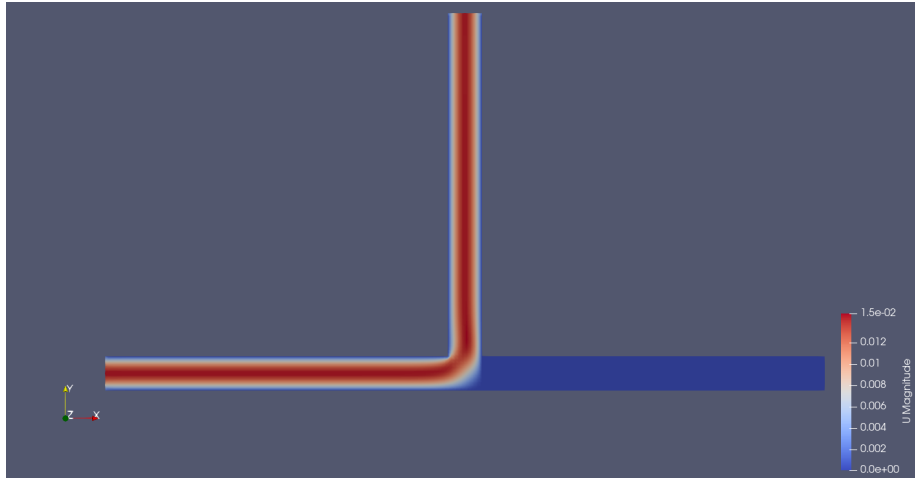
In the membrane gas separation module, the membrane is represented as an **internal permeable wall**.

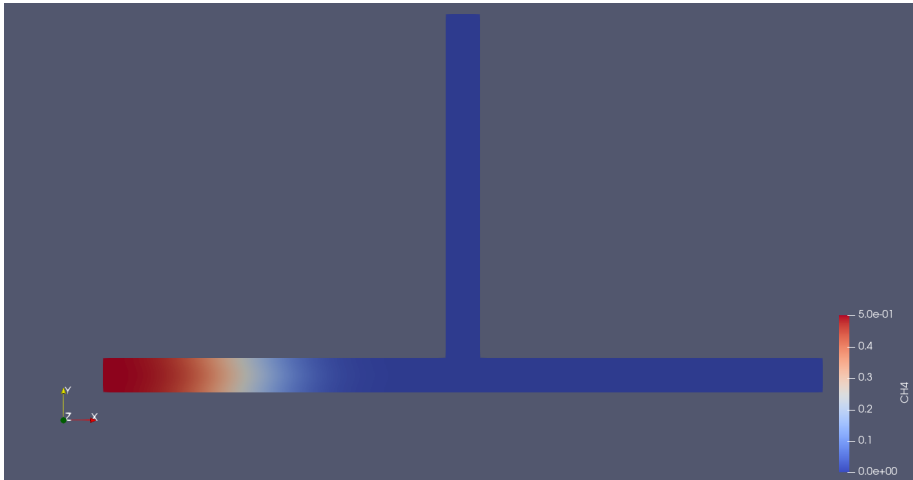
To simulate this, we will use the OpenFOAM utility **baffle**.

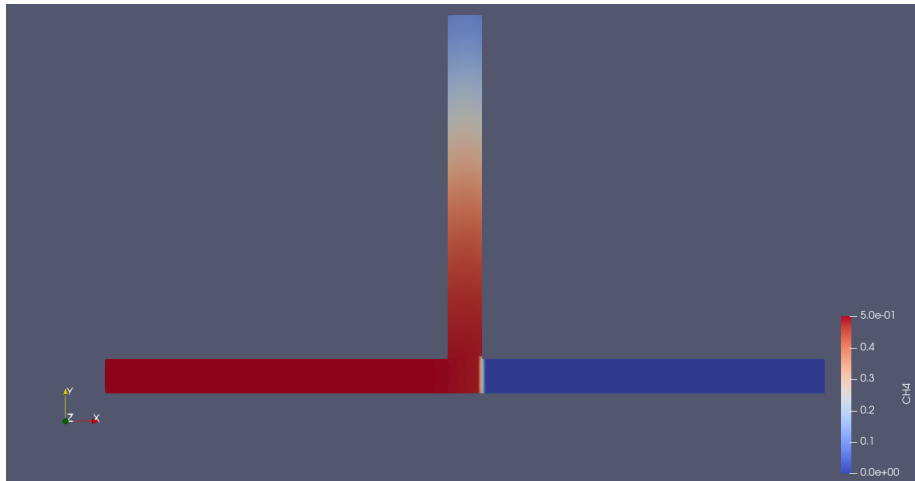
In Lecture 01, we used a baffle as a **solid wall** (non-permeable membrane). Simulation 01 (next slide) revisits this concept by including a baffle acting as a solid wall.

Starting from Simulation 02, we will configure the baffle to behave as a **permeable membrane**.

Simulation 01: Gas in a 2D T-Junction with a solid wall.







To create this internal wall, first we need to include the membrane in the msh file. You can find the membrane surface defined in the *tJunction.geo* file (from gmsh).

Then we need to execute the command *createBaffles*.

The configuration for the *createBaffles* command can be found in the *createBafflesDict* file, in the system folder.

system/createBafflesDict

```
1  internalFacesOnly true;
2  baffles
3  {
4      membrane
5      {
6          type          faceZone;
7          zoneName      membrane;
8          owner
9          {
10             name       membraneLeft;
11             type        mappedWall;
12             neighbourPatch membraneRight;
13         }
14         neighbour
15         {
16             name       membraneRight;
17             type        mappedWall;
18             neighbourPatch membraneLeft;
19         }
20     }
21 }
```


The file `system/createBaffles` is used to convert internal faces into boundary patches.

internalFacesOnly true; This means only internal faces are considered for conversion.

baffles {} Defines the set of internal faces to convert into baffles. Each entry corresponds to one face zone.

membrane {} Creates baffles from the faceZone named **membrane**.

owner and **neighbour** define the two sides of the baffle:

- **name**: patch name for each side.
- **type**: both are set as **mappedWall**.
- **neighbourPatch**: links each patch to the other.

This creates a pair of linked walls, useful for simulating internal structures, like internal walls or membranes.

We need to include the *membrane* patch in **every file** in the *0* folder.

Example: *0/U*.

```
1  ...
2
3  membrane
4  {
5      type          noSlip;
6  }
7
8  ...
```

Now we can run the simulation.

Take a look at the *scriptRun.py* file for all the commands.

Sumário

- 1 Gas Flow Simulation
- 2 Baffles (Membranes)
- 3 Simulation: T-Junction with Membrane**
- 4 SnappyHexMesh
- 5 Mesh Generation with SHM
- 6 Simulation: 3D T-Junction with Membrane and SHM
- 7 Conclusion

How can we make the baffle or internal wall behave like a **permeable membrane**?

We achieve this by applying **special boundary conditions** to all variables.

For species concentration, we will use the boundary condition type `semiPermeableBaffleMassFraction`.

```
1 dimensions      [0 0 0 0 0 0 0];
2
3 internalField uniform 0.0;
4
5 boundaryField
6 {
7
8     membraneLeft
9     {
10         type          semiPermeableBaffleMassFraction;
11         samplePatch    membraneRight;
12         c              1e-8;
13         property       massFraction;
14         value          $internalField;
15     }
16
17     ...
```

type: semiPermeableBaffleMassFraction

Applies a boundary condition that allows selective mass transfer based on mass fraction.

samplePatch: membraneRight

Specifies the opposite side of the membrane used for sampling values.

c: 1e-8

Permeability coefficient (controls the flux across the membrane).

property: massFraction

Indicates that the condition is applied to the mass fraction of species.

value: \$internalField

Initial guess taken from the internal field.

Source code: *semiPermeableBaffleMassFraction.H* .

```
1  Class
2      Foam::semiPermeableBaffleMassFractionFvPatchScalarField
3
4  Description
5      This is a mass-fraction boundary condition for a semi-permeable baffle.
6
7      This condition models a baffle which is permeable to a some species and
8      impermeable to others. It must be used in conjunction with a
9      specieTransferVelocityFvPatchVectorField velocity condition, and a
10     specieTransferTemperatureFvPatchScalarField temperature condition.
```

The mass flux of a species is calculated as a coefficient multiplied by the difference in a property across the baffle.

```
\f[
  \phi_{Yi} = c A (\psi_i - \psi_{i,n})
\f]
where
\variable
  \phi_{Yi} | Flux of the permeable specie [kg/s]
  c         | Transfer coefficient [kg/m^2/s/<property-dimensions>]
  A         | Patch face area [m^2]
  \psi_i    | Property on the patch [<property-dimensions>]
  \psi_{i,n} | Property on the neighbour patch [<property-dimensions>]
\endvariable
```

A species that the baffle is permeable to will, therefore, have a coefficient greater than zero, whilst a species that does **not** transfer will have a coefficient equal to zero.

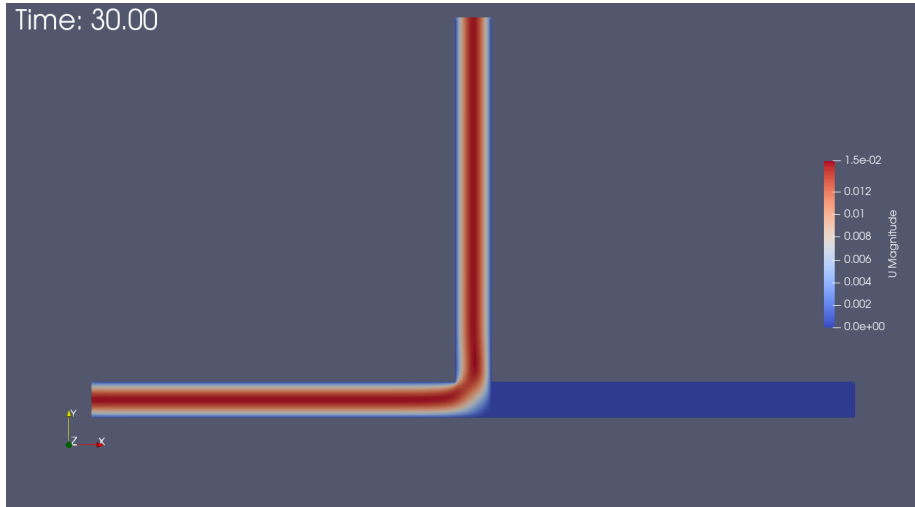
Usage

```
\table
  Property | Description          | Req'd? | Default
  c        | Transfer coefficient        | no      | 0
  property | Property used to drive the transfer; massFraction, \\\
moleFraction, molarConcentration, or partialPressure | if c is \\\
non-zero |
  phi      | Name of the flux field      | no      | phi
  U        | Name of the velocity field  | no      | U
\endtable
```

Example of the boundary condition specification:

```
\verbatimim
<patchName>
{
  type          semiPermeableBaffleMassFraction;
  property      molarConcentration;
  c             1e-3;
  value         $internalField;
}
\endverbatimim
```

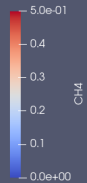
Simulation 02: Gas in a 2D T-Junction with a membrane.



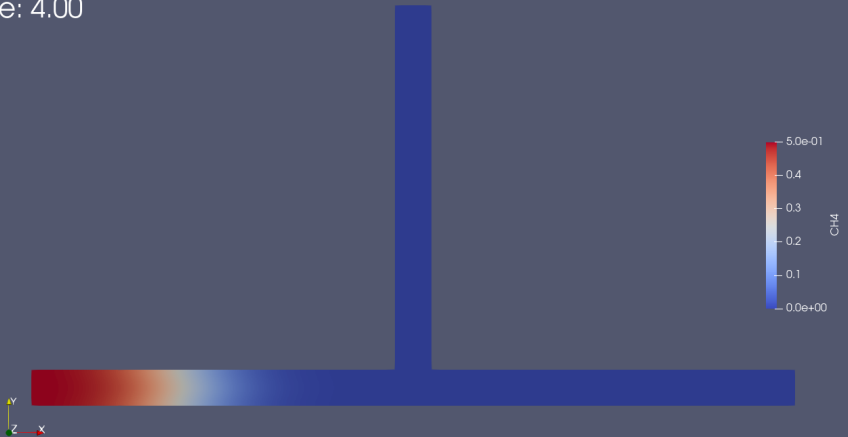
Time: 0.00



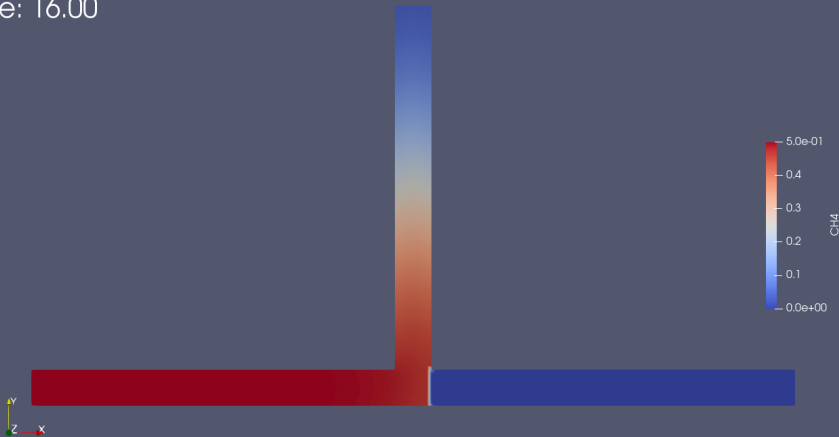
Time: 0.00



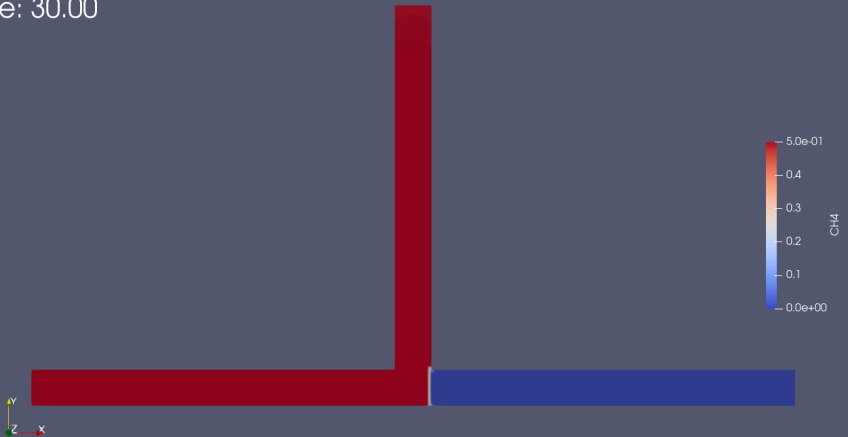
Time: 4.00



Time: 16.00



Time: 30.00

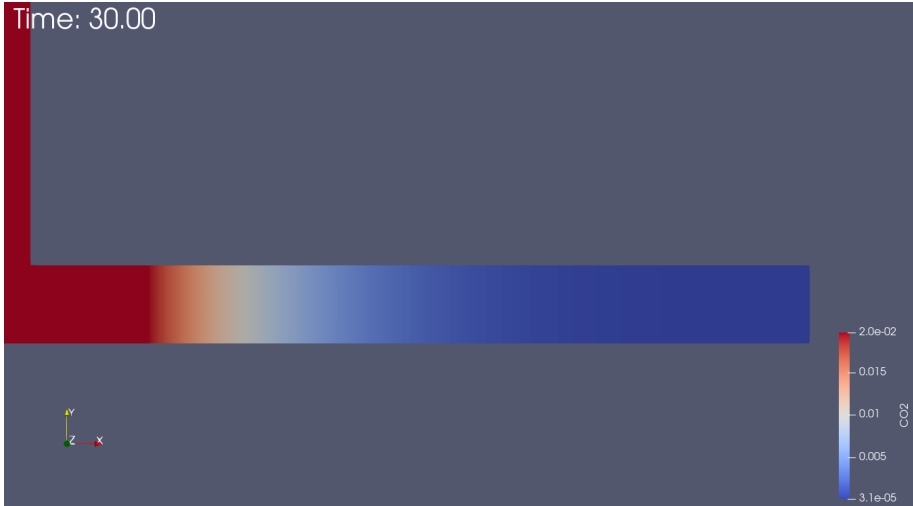


Note that there is now some flux across the membrane. This flux is regulated by the transfer coefficient (c).

Since c is greater for CO₂, a larger fraction of CO₂ permeates through the membrane and appears on the permeate side.



Time: 30.00



Velocity magnitude in the permeate side.



To run the simulation, we generate the mesh with the command *gmshToFoam*.

Then we include the membrane with the command *createBaffles*.

We also need to execute the command *setFields*, to set the initial condition for the simulation.

Excerpt from the *setFieldsDict* file.

```
1 defaultFieldValues
2 (
3     //volScalarFieldValue CH4 0.0
4     //volScalarFieldValue CO2 0.0
5     //volScalarFieldValue N2 1.0
6 );
7 regions
8 (
9     zoneToCell
10    {
11        zone permeate;
12        fieldValues
13        (
14            //volScalarFieldValue CH4 1e-4
15            volScalarFieldValue CO2 1.e-10
16            //volScalarFieldValue N2 1.0
17        );
18    }
19 );
```

Copy all field files from the 0 folder to the latest time folder, then execute the command:

```
$ setFields -latestTime
```

The option `-latestTime` tells OpenFOAM to apply the command to the folder with the highest time value.

Important Note: the value of C_{O2} in the permeate zone should be zero. However, we set it to $1e-10$.

If the initial value is exactly zero, the simulation fails with a memory error. Although the exact cause is unclear, it is likely due to a division by zero when the concentration is zero throughout the region at the start of the simulation.

Using $1e-10$ avoids the error and does not affect the results.

In summary, these are the steps to run the simulation (*scriptRun.py*):

- 1 Generate the geometry and the mesh (with the membrane physical surface) using Gmsh.
- 2 Convert the mesh using the command `gmshToFoam`.
- 3 Change the type of the *frontAndBack* (to *empty*) and *walls* (to *wall*) patches in the *constant/polyMesh/boundary* file.
- 4 Create the membrane using the command `createBaffles`.
- 5 Copy the field files (`CH4`, `CO2`, `N2`, `p`, `T`, `U`) from the 0 folder to the folder with the highest time value.
- 6 Define the initial condition using the command `setFields`.
- 7 Run the simulation using the command `foamRun`.

This is the membrane that we are going to use in the membrane gas separation module.

Note that the membrane allows the passage of CH_4 , CO_2 and N_2 , but with different transfer coefficients.

Since c (transfer coefficient) is greater for CO_2 , it allows more CO_2 to permeate. As a result, the retentate side becomes enriched in CH_4 , leading to a **gas stream with higher CH_4 concentration at the outlet**.

This is how our membrane works. However, the actual membrane operates based on a difference in **partial pressure**, not mass fraction. We will correct this in the next lecture.

Sumário

- 1 Gas Flow Simulation
- 2 Baffles (Membranes)
- 3 Simulation: T-Junction with Membrane
- 4 SnappyHexMesh**
- 5 Mesh Generation with SHM
- 6 Simulation: 3D T-Junction with Membrane and SHM
- 7 Conclusion

This section introduces the mesh generation utility ***snappyHexMesh*** (SHM), included with OpenFOAM.

The ***snappyHexMesh*** utility generates three-dimensional meshes containing hexahedrons (*hex*) and split hexahedrons (*split-hex*) automatically from triangulated surface geometries, or *tri-surfaces*.

To run ***snappyHexMesh***, you need to have a geometry in **STL** (Stereolithography) or OBJ (Wavefront Object) format.

Generating the mesh with *snappyHexMesh* is a **simulation in itself** and can take hours, depending on the number of cells and the complexity of the mesh.

Several **iterations** are performed until the mesh conforms to the input geometry, based on the specified parameters.

An optional phase will shrink the resulting mesh and insert cell layers around the surfaces.

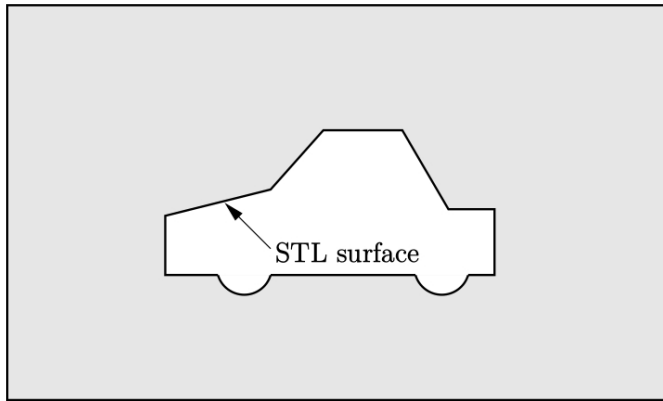
The mesh refinement level is highly customizable, and surface handling is robust, with a pre-specified final mesh quality.

To learn how to use *snappy*, I recommend:

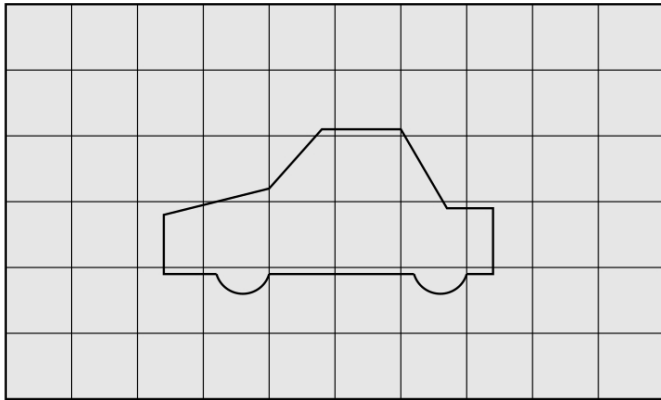
- The official *User Guide*;
- The *Wolf Dynamics* slides (They're excellent!! I'll share them with you.);
- OpenFOAM tutorials (examples: *incompressibleFluid/wingMotion*, *incompressibleFluid/motorBike*, and *multicomponentFluid/membrane*).

What does *snappyHexMesh* do?

It takes the geometry you created (STL or OBJ).



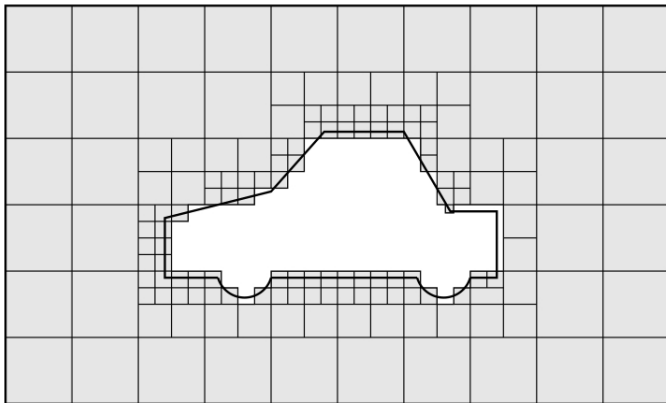
And a background mesh, made by hexahedrons (usually made with *blockMesh*).



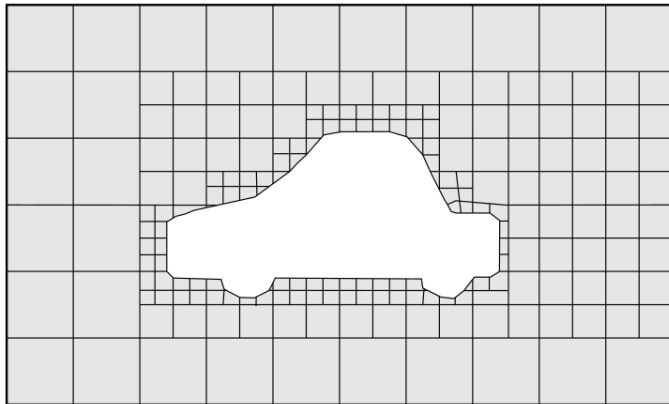
The first step involves identifying cells that are inside (or outside, depending on your configuration) of the object.



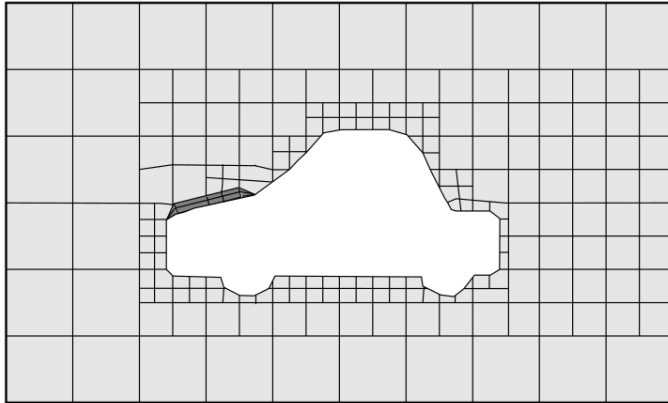
And then these cells are removed. This step is called ***castellate***.



The next step is called ***snap*** and consists of attaching the cells to the surface, better adapting the mesh to the geometry.



The final step, which is actually optional, involves adding cell layers to surfaces. This step is called ***addLayers***.



In summary, there are 3 steps:

- *castellate*
- *snap*
- *addLayers*

These 3 steps have numerous configuration parameters.

To generate a mesh with `snappyHexMesh`, we will need:

- a very simple background mesh composed of hexahedrons (we'll use *blockMesh* to generate this mesh);
- an *stl* file with the geometry around which or inside which we want to generate the mesh (we'll use *Gmsh* to create this geometry);
- a configured *snappyHexMeshDict* file.

In the next slides we'll see an example.

Sumário

- 1 Gas Flow Simulation
- 2 Baffles (Membranes)
- 3 Simulation: T-Junction with Membrane
- 4 SnappyHexMesh
- 5 Mesh Generation with SHM**
- 6 Simulation: 3D T-Junction with Membrane and SHM
- 7 Conclusion

Simulation 03: Gas in a pipe with snappyHexMesh.

We need three things to generate the mesh with *snappyHexMesh*:

- a background mesh generated with *blockMesh*;
- a stl geometry file generated with *gms*h;
- and a *snappyHexMeshDict* file to configure the mesh properties.

Let's start with the background mesh. To generate this mesh, we'll use *blockMesh*.

It's important (for quality purposes) that the cells have an aspect ratio close to 1.

It's also crucial to have cells that intersect with the geometry. The background mesh must be bigger than the stl geometry.

Note: Since our problem is $3D$, I'll use a coarse mesh for faster simulation. (To improve this mesh, you could add more points in the *blockMeshDict*.)

blockMeshDict file:

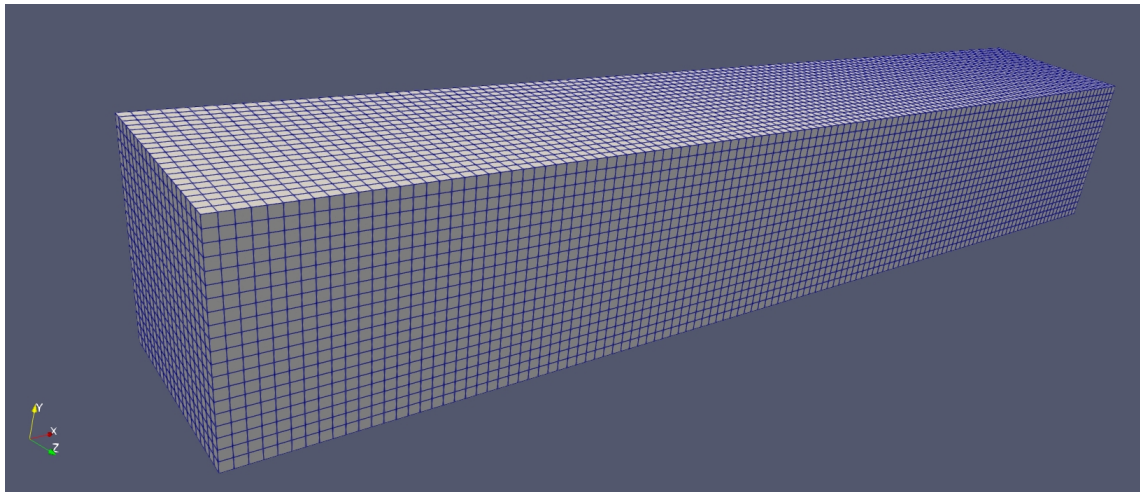
```
1  convertToMeters 1.0;
2
3  vertices
4  (
5      (-0.01 -0.01 -0.01) // 0
6      (0.11 -0.01 -0.01)  // 1
7      (0.11 0.01 -0.01)   // 2
8      (-0.01 0.01 -0.01)  // 3
9      (-0.01 -0.01 0.01)  // 4
10     (0.11 -0.01 0.01)   // 5
11     (0.11 0.01 0.01)    // 6
12     (-0.01 0.01 0.01)   // 7
13 );
14
15 blocks
16 (
17     hex (0 1 2 3 4 5 6 7) (120 20 20) simpleGrading (1 1 1)
18 );
```

```
19 boundary
20 (
21     inlet
22     {
23         type patch;
24         faces
25         (
26             (0 4 7 3)
27         );
28     }
29     outlet
30     {
31         type patch;
32         faces
33         (
34             (2 6 5 1)
35         );
36     }
```

```
37     topWall
38     {
39         type wall;
40         faces
41         (
42             (6 2 3 7)
43         );
44     }
45     bottomWall
46     {
47         type wall;
48         faces
49         (
50             (0 1 5 4)
51         );
52     }
```

```
53     frontAndBack
54     {
55         type wall;
56         faces
57         (
58             (0 3 2 1)
59             (4 5 6 7)
60         );
61     }
62 );
```

Now just run the *blockMesh* command to generate our background mesh.



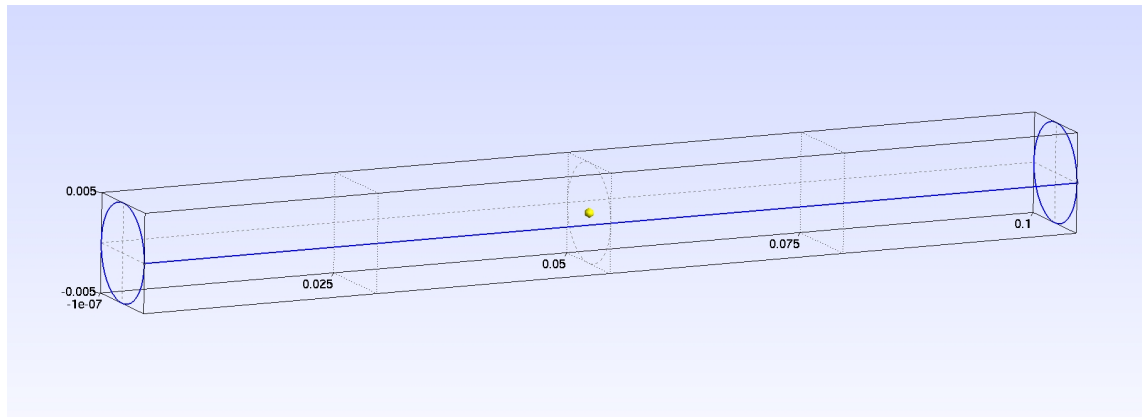
Now we'll generate our geometry, which is the *stl* file.

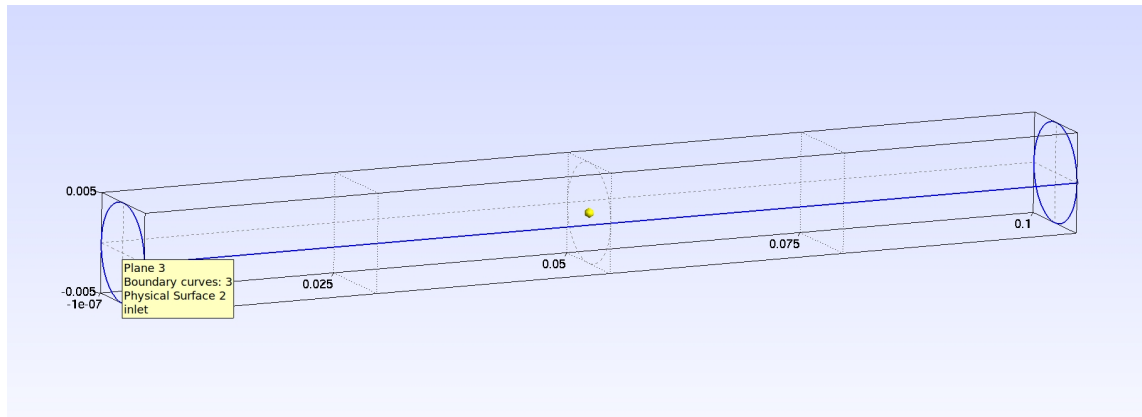
For this we'll use *Gmsh*.

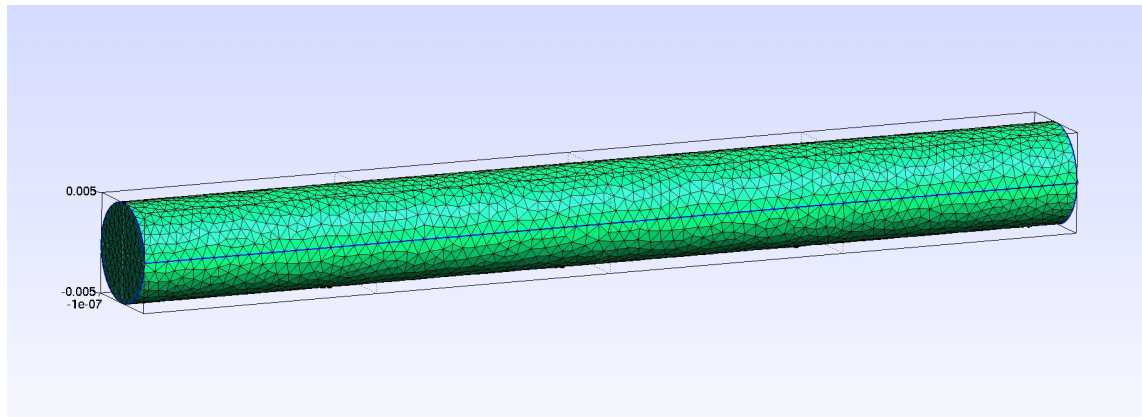
The code used to generate the geometry is in the following slides.

Pipe *pipe.geo*:

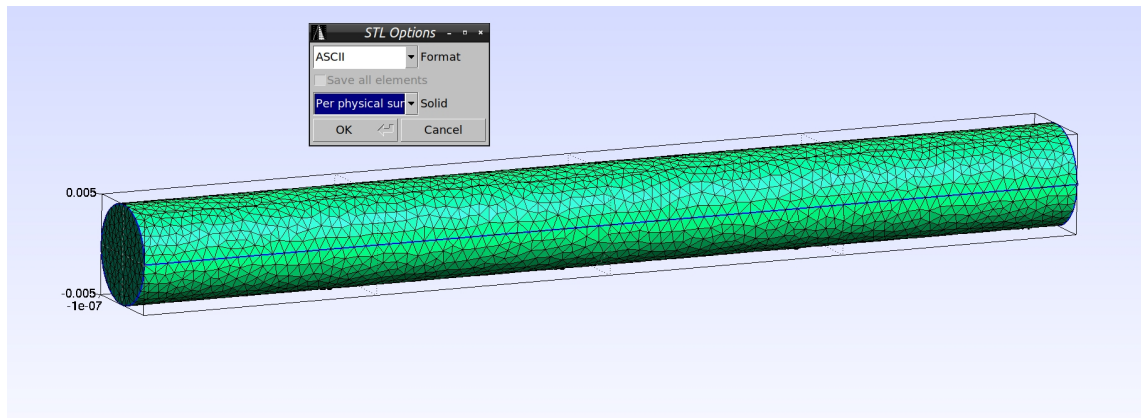
```
1 // Gmsh project created on Tue May 20 13:04:01 2025
2 SetFactory("OpenCASCADE");
3 //+
4 Cylinder(1) = {0, 0, 0, 0.1, 0, 0, 0.005, 2*Pi};
5 //+
6 Physical Volume("internal") = {1};
7 //+
8 Physical Surface("inlet") = {3};
9 //+
10 Physical Surface("wall") = {1};
11 //+
12 Physical Surface("outlet") = {2};
13 //+
14 Mesh.Algorithm = 6;
15 //+
16 Mesh 5;
17 //+
18 RefineMesh;
19 RefineMesh;
20 RefineMesh;
```





Export as a *stl* file, with the “Per physical surface” option.



Note that now we are not generating a mesh for *OpenFOAM*. We are generating a surface geometry that *snappyHexMesh* will use to generate a mesh.

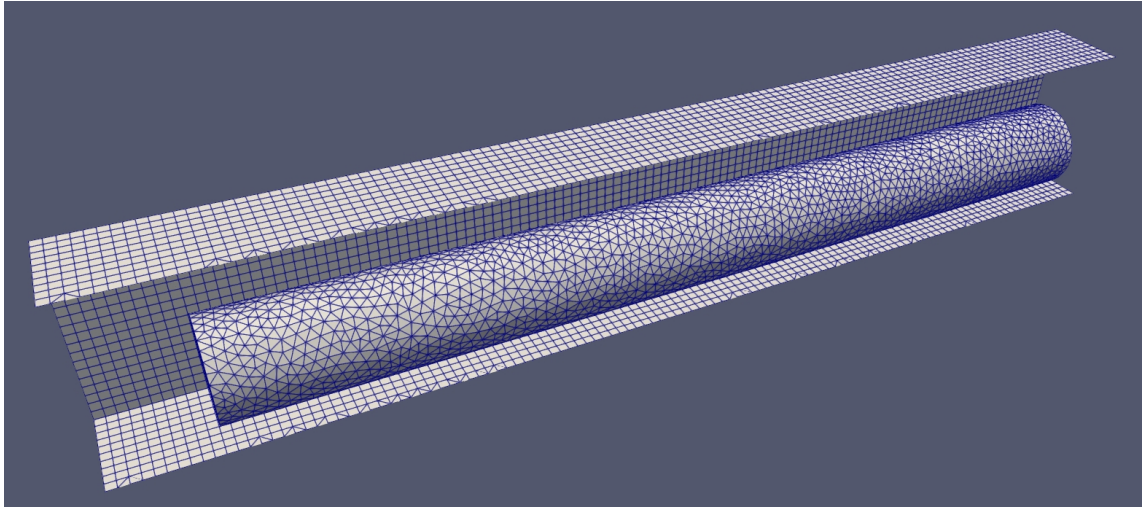
This *RefineMesh* at the end is important to generate a high-quality *stl* file, which ensures better geometry detail.

In *Gmsh*, export the geometry as *stl*.

Save in *ASCII* format with the ***Per physical surface*** option.

Place the *pipe.stl* file in the folder *constant/geometry* (create the folder, if it does not exist).

The geometry must be inside the background mesh.



We have the background mesh and the *stl* geometry file. Now let's proceed to the main file: *snappyHexMeshDict*.

The key entries in the file are:

- castellatedMesh
- snap
- addLayers
- geometry
- castellatedMeshControls
- snapControls
- addLayersControls
- meshQualityControls

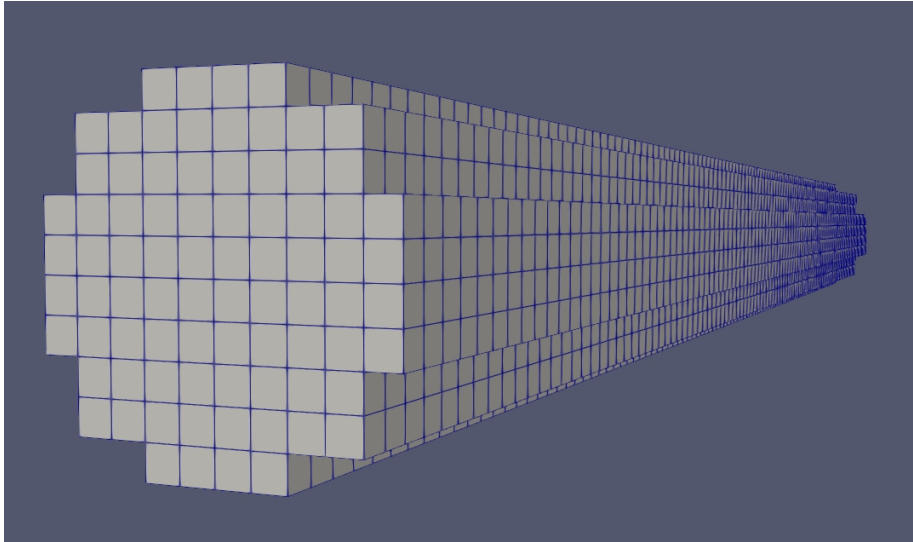
We'll examine the some parts of the file.

File *snappyHexMeshDict* .

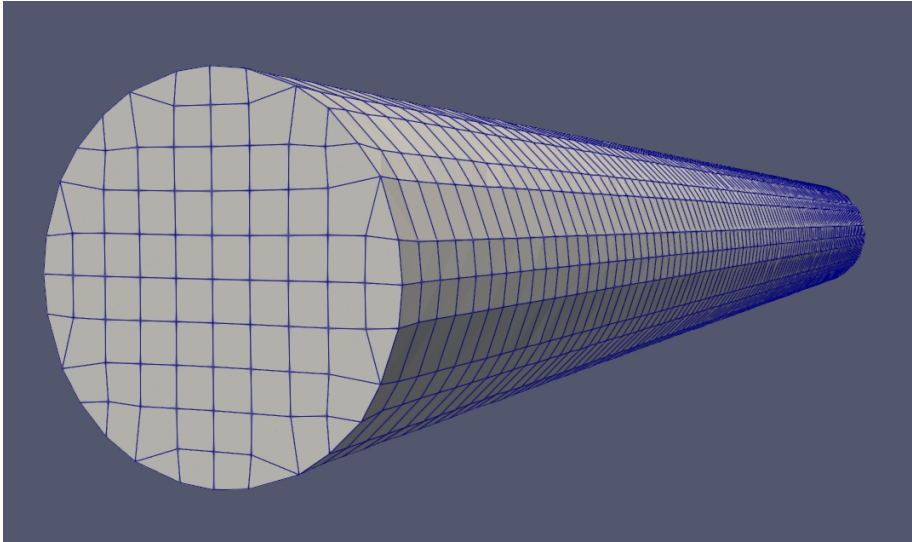
```
1 FoamFile
2 {
3     format      ascii;
4     class       dictionary;
5     object      snappyHexMeshDict;
6 }
7 // * * * * *
8
9 // Which of the steps to run
10 castellatedMesh true;
11 snap           true;
12 addLayers      true;
```

First we choose which steps to execute. We can run just *castellatedMesh*. Or run *castellatedMesh* once and then modify *snap* and *addLayers* parameters, executing only these two subsequently without needing to rerun *castellatedMesh* each time.

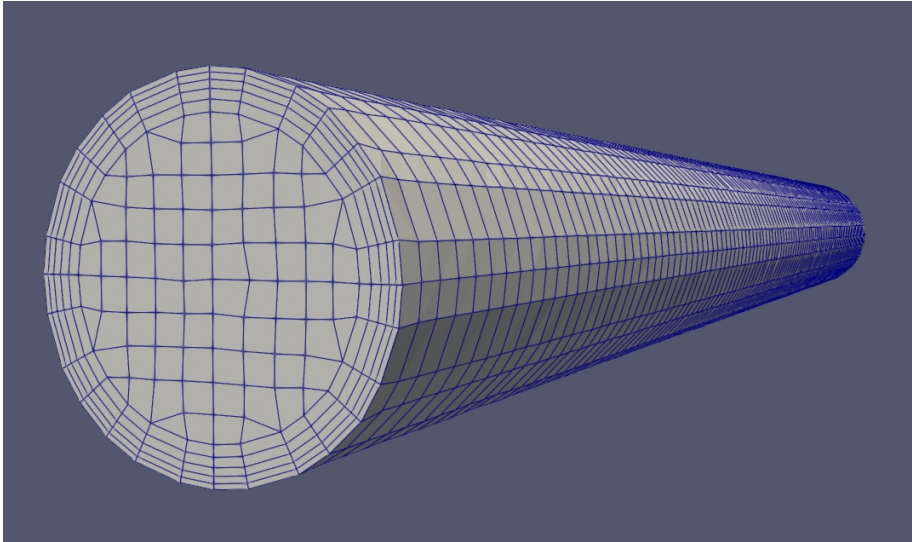
The *castellatedMesh* step does this:



The *snap* step does this:



The *addLayers* step does this:



We'll keep all 3 options *true*. Continuing with the code, let's configure the geometry section.

```
13 geometry
14 {
15     pipe
16     {
17         type triSurfaceMesh;
18         file "pipe.stl";
19         regions
20         {
21             inlet      {name inlet;}
22             outlet      {name outlet;}
23             wall         {name wall;}
24         }
25     }
26 };
```

Note that the file itself contains various explanations about the parameters. I took this file from the *incompressibleFluid/motorBike* tutorial.

I will discuss here the most important parameters. Some parameters I will keep as it is in the tutorial.

The *geometry* dictionary is where we specify which STL file to use.

The surface names are given in the *geometry* dictionary.

With the geometry defined, we'll now proceed to *castellatedMeshControls*, which stores the parameters for the *castellatedMesh* step.

File *snappyHexMeshDict*.

```
27 castellatedMeshControls
28 {
29     // Refinement parameters
30     // ~~~~~
31
32     // If local number of cells is >= maxLocalCells on any processor
33     // switches from from refinement followed by balancing
34     // (current method) to (weighted) balancing before refinement.
35     maxLocalCells 100000;
36
37     // Overall cell limit (approximately). Refinement will stop immediately
38     // upon reaching this number so a refinement level might not complete.
39     // Note that this is the number of cells before removing the part which
40     // is not 'visible' from the keepPoint. The final number of cells might
41     // actually be a lot less.
42     maxGlobalCells 7000000;
```

The *maxLocalCells* parameter defines the maximum number of cells a parallel process can have during local mesh refinement.

During snappyHexMesh execution, the mesh is refined locally in specific geometry regions requiring higher detail for CFD simulation.

The *maxGlobalCells* parameter sets the total maximum number of cells allowed after global and local refinement during mesh generation.

It controls the maximum cell count in the global mesh domain, including all locally refined regions.

```
43 refinementSurfaces
44 {
45     pipe
46     {
47         // Surface-wise min and max refinement level
48         level (1 1);
49
50         regions
51         {
52
53             inlet
54             {
55                 level (0 0);
56                 patchInfo
57                 {
58                     type patch;
59                 }
60             }
```



```

61         outlet
62         {
63             level (0 0);
64             patchInfo
65             {
66                 type patch;
67             }
68         }
69
70         wall
71         {
72             level (0 0);
73             patchInfo
74             {
75                 type patch;
76             }
77         }
78     }
79
80 }
81

```

The *level* determines how many times the mesh is refined on specified surfaces.

Higher level values produce finer mesh in these locations.

The *type* of the surface is defined in the *patchInfo* dictionary. The default is *wall*. This type is very important to define the **boundary conditions**.

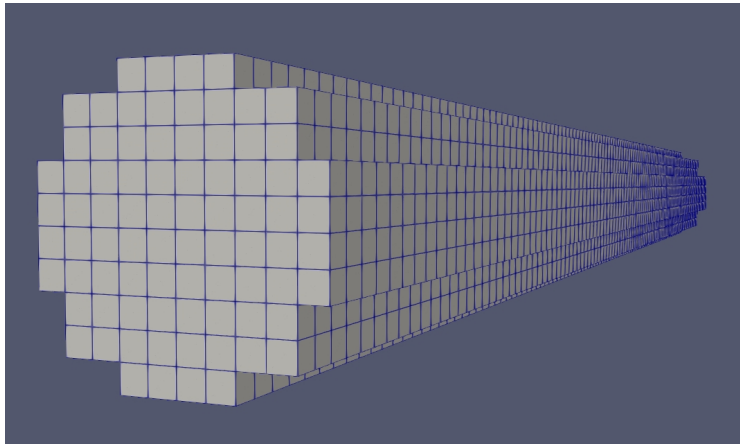
```
82 // Mesh selection
83 // ~~~~~
84
85 // After refinement patches get added for all refinementSurfaces and
86 // all cells intersecting the surfaces get put into these patches. The
87 // section reachable from the insidePoint is kept.
88 // NOTE: This point should never be on a face, always inside a cell, even
89 // after refinement.
90 insidePoint (0.00001 0.0 0.0);
91 }
```

Important: the *insidePoint* parameter is very important.

This point defines whether the mesh will be generated inside or outside the specified geometry.

Ensure this point is never on a face and in the correct side.

This is the *castellatedMesh* result. Note the region with finer refinement downstream of the cylinder - this is the *refinementBox* we created.



Now let's configure the *snap* phase parameters.

```
92 snapControls
93 {
94     //- Number of patch smoothing iterations before finding correspondence
95     // to surface
96     nSmoothPatch 3;
97
98     //- Relative distance for points to be attracted by surface feature point
99     // or edge. True distance is this factor times local
100    // maximum edge length.
101    tolerance 5.0;
102
103    //- Number of mesh displacement relaxation iterations.
104    nSolveIter 30;
105
106    //- Maximum number of snapping relaxation iterations. Should stop
107    // before upon reaching a correct mesh.
108    nRelaxIter 5;
```

```
109 // Feature snapping
110
111 //- Number of feature edge snapping iterations.
112 // Leave out altogether to disable.
113 nFeatureSnapIter 5;
114
115 //- Detect (geometric) features by sampling the surface
116 implicitFeatureSnap false;
117
118 //- Use castellatedMeshControls::features
119 explicitFeatureSnap true;
120
121 //- Detect features between multiple surfaces
122 // (only for explicitFeatureSnap, default = false)
123 multiRegionFeatureSnap true;
124
125 }
```

After *snap*, comes *addLayers*.

addLayers controls adding cell layers near geometry surfaces.

These additional cell layers are often used to improve near-surface resolution and ensure accurate capture of flow effects or other physical interactions.

The process typically involves multiple iterations to ensure proper mesh refinement and meet specified quality criteria.

addLayers parameters.

```
126 addLayersControls
127 {
128     // Are the thickness parameters below relative to the undistorted
129     // size of the refined cell outside layer (true) or absolute sizes (false).
130     relativeSizes true;
131
132     // Per final patch (so not geometry!) the layer information
133     layers
134     {
135         wall
136         {
137             nSurfaceLayers 5;
138         }
139     }
140
141     // Expansion factor for layer mesh
142     expansionRatio 1.2;
```

```
143 // Wanted thickness of final added cell layer. If multiple layers
144 // is the thickness of the layer furthest away from the wall.
145 // Relative to undistorted size of cell outside layer.
146 // See relativeSizes parameter.
147 finalLayerThickness 0.3;
148
149 // Minimum thickness of cell layer. If for any reason layer
150 // cannot be above minThickness do not add layer.
151 // Relative to undistorted size of cell outside layer.
152 minThickness 0.1;
153
154 // If points get not extruded do nGrow layers of connected faces that are
155 // also not grown. This helps convergence of the layer addition process
156 // close to features.
157 nGrow 0;
```

```
158 // Advanced settings
159
160 // When not to extrude surface. 0 is flat surface, 90 is when two faces
161 // are perpendicular
162 featureAngle 60;
163
164 // Maximum number of snapping relaxation iterations. Should stop
165 // before upon reaching a correct mesh.
166 nRelaxIter 3;
167
168 // Number of smoothing iterations of surface normals
169 nSmoothSurfaceNormals 1;
170
171 // Number of smoothing iterations of interior mesh movement direction
172 nSmoothNormals 3;
173
174 // Smooth layer thickness over surface patches
175 nSmoothThickness 2;
```

```
176 // Stop layer growth on highly warped cells
177 maxFaceThicknessRatio 0.5;
178
179 // Reduce layer growth where ratio thickness to medial
180 // distance is large
181 maxThicknessToMedialRatio 0.3;
182
183 // Angle used to pick up medial axis points
184 minMedianAxisAngle 90;
185
186 // Create buffer region for new layer terminations
187 nBufferCellsNoExtrude 0;
188
189 // Overall max number of layer addition iterations
190 nLayerIter 50;
191 }
```

Finally, we can define desired mesh quality parameters in the *meshQualityControls* dictionary.

```
1 // Generic mesh quality settings.
2 meshQualityControls
3 {
4     //- Maximum non-orthogonality allowed. Set to 180 to disable.
5     maxNonOrtho 65;
6
7     //- Max skewness allowed. Set to <0 to disable.
8     maxBoundarySkewness 20;
9     maxInternalSkewness 4;
10
11     //- Max concaveness allowed. Is angle (in degrees) below which concavity
12     // is allowed. 0 is straight face, <0 would be convex face.
13     // Set to 180 to disable.
14     maxConcave 80;
15
16     //- Minimum projected area v.s. actual area. Set to -1 to disable.
17     minFlatness 0.5;
18
19     ...
20 }
```

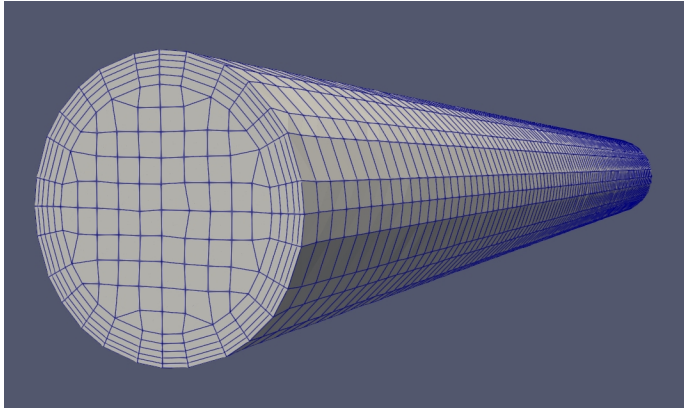
Ready! Run the command:

```
$ snappyHexMesh
```

Three new directories will appear, each containing meshes from the *castellate*, *snap*, and *addLayers* processes.

It's useful to examine all three stages when developing your mesh and parameters.

Important: The folders with the mesh will be named as time steps, but the simulation was not executed. This is just a way that OpenFOAM uses to create a new mesh without deleting the original one.



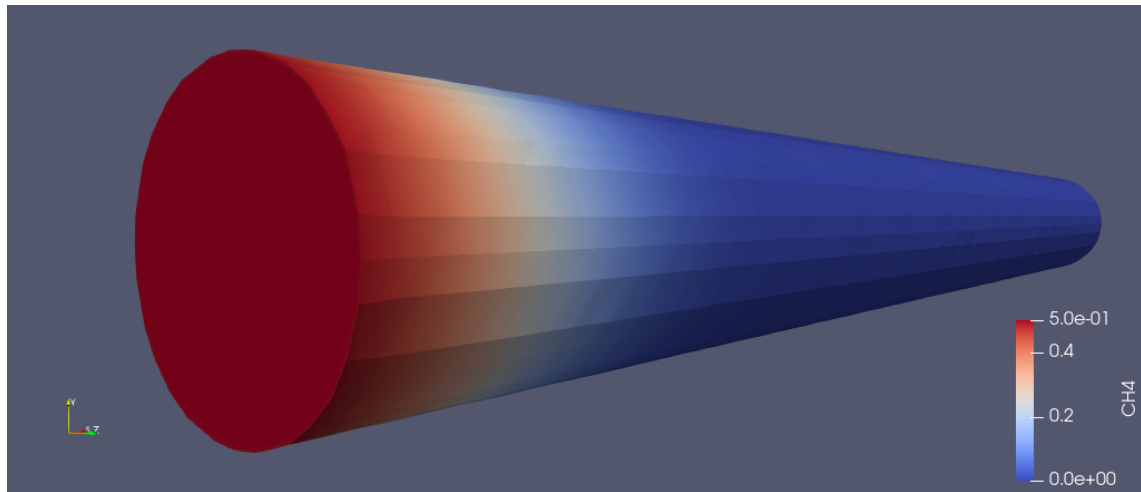
Step and command summary:

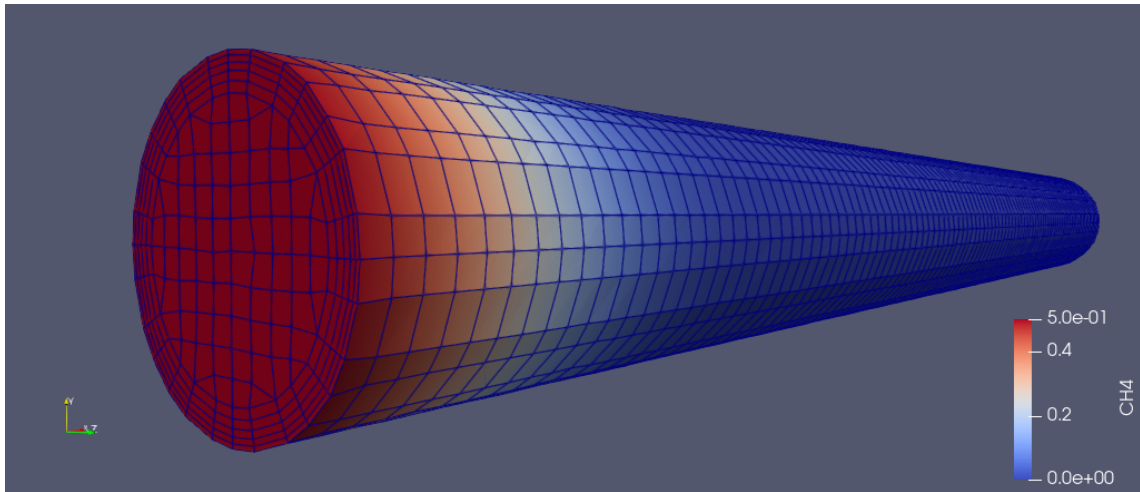
- Run *blockMesh*
- Open *pipe.geo* and save as *pipe.stl*
- Run *surfaceFeatures*
- Run *snappyHexMesh*

Now you need to copy the field files from 0 folder to the last folder.

In the *scripRun.py* file you can find all the steps to run the simulation.

Now we can run the simulation with the command ***foamRun*** .





Conclusion (from snappyHexMesh part).

Modify parameters and generate other meshes. Study each parameter's influence on the final mesh.

Create other geometries and generate their respective meshes. Run complete test cases.

snappyHexMesh is a powerful mesh generator worth mastering for projects involving complex geometries.

Study the *Wolf Dynamics* slides about *SHM*.

Conclusion – snappyHexMesh

- *snappyHexMesh* is a powerful and flexible tool for meshing complex geometries in OpenFOAM.
- Mastering its parameters requires practice: change the settings, observe results, and compare mesh quality.
- Experiment with different STL geometries and boundary conditions to build confidence.
- Use visualization tools (e.g., ParaView) to inspect each mesh stage: *castellated*, *snapped*, and *addLayers*.
- Refer to the *Wolf Dynamics* training material for deeper insights and practical examples.

Sumário

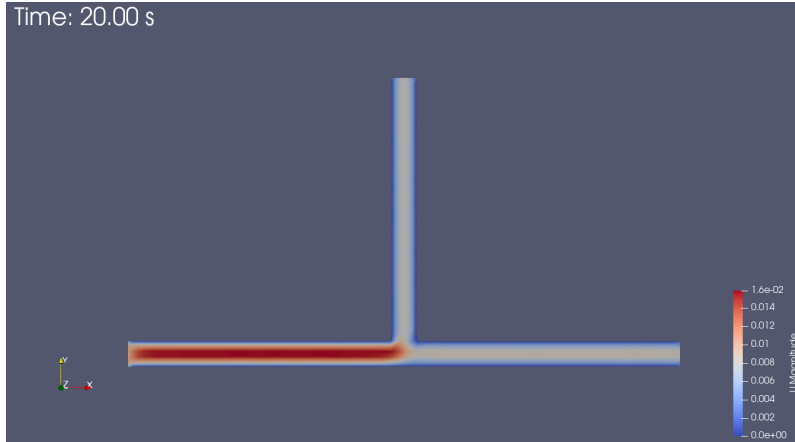
- 1 Gas Flow Simulation
- 2 Baffles (Membranes)
- 3 Simulation: T-Junction with Membrane
- 4 SnappyHexMesh
- 5 Mesh Generation with SHM
- 6 Simulation: 3D T-Junction with Membrane and SHM**
- 7 Conclusion

Now let's return to our T-Junction simulations.

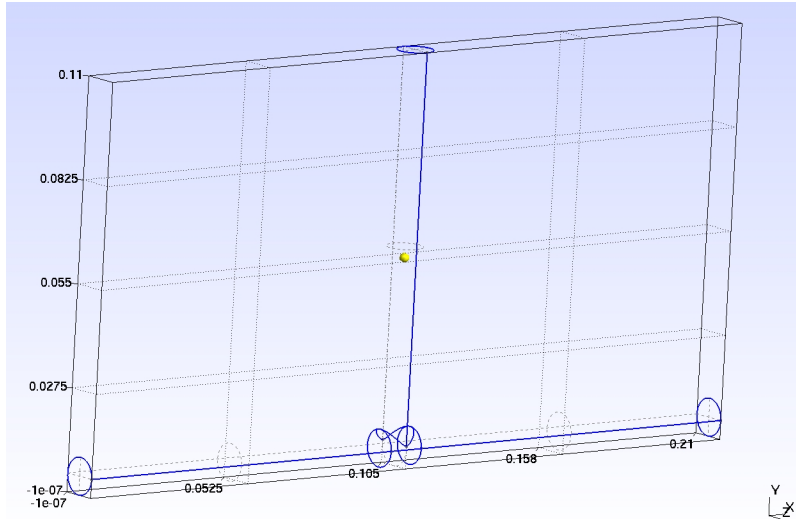
In **Simulation 4**, we will investigate the gas flow in a T-Junction with a circular cross-section, with no membrane.

Then, in **Simulation 5**, we conclude this lecture with the complete case: gas flow in a 3D T-Junction with a circular cross-section, including a selective membrane.

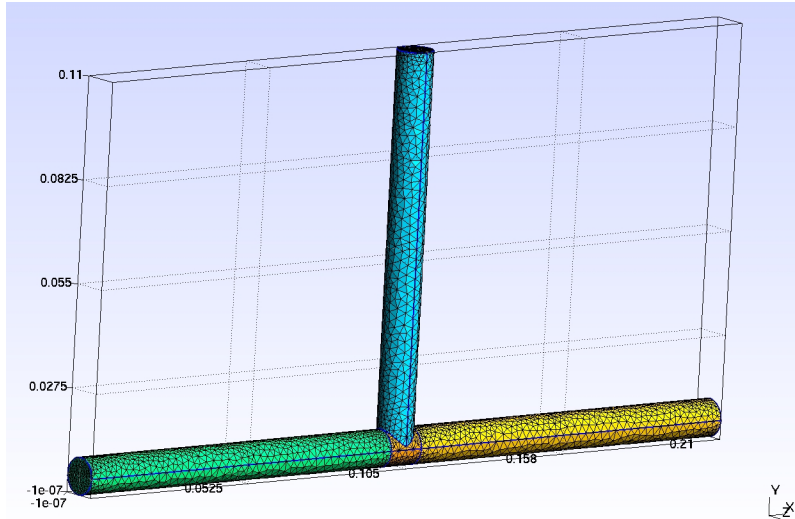
Simulation 04: Gas in a T-Junction with snappyHexMesh.



Geometry.

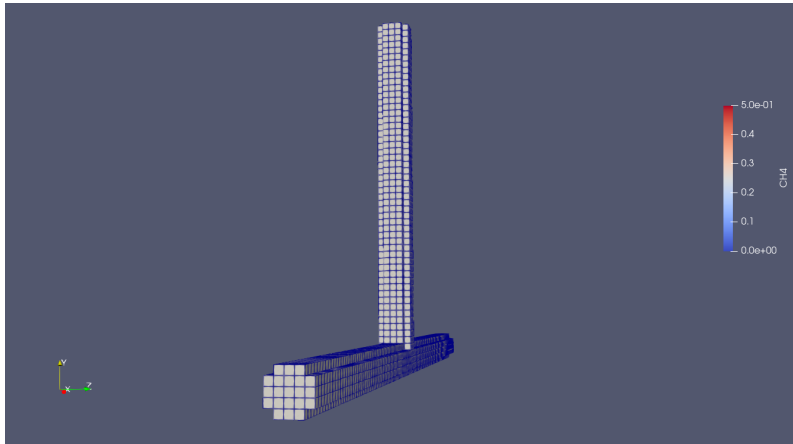


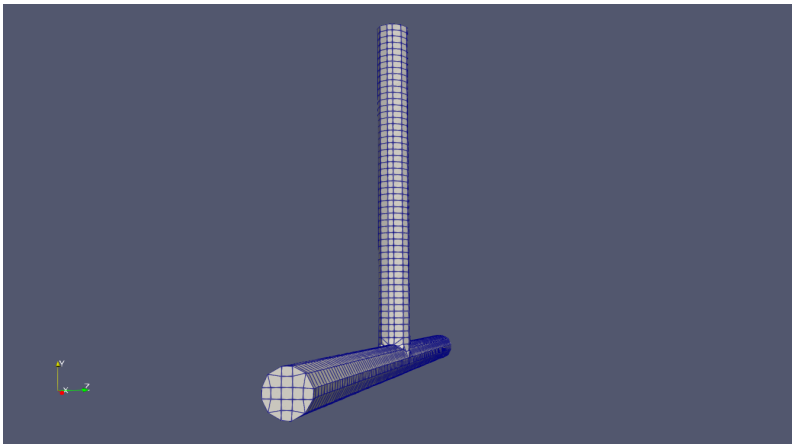
Geometry.

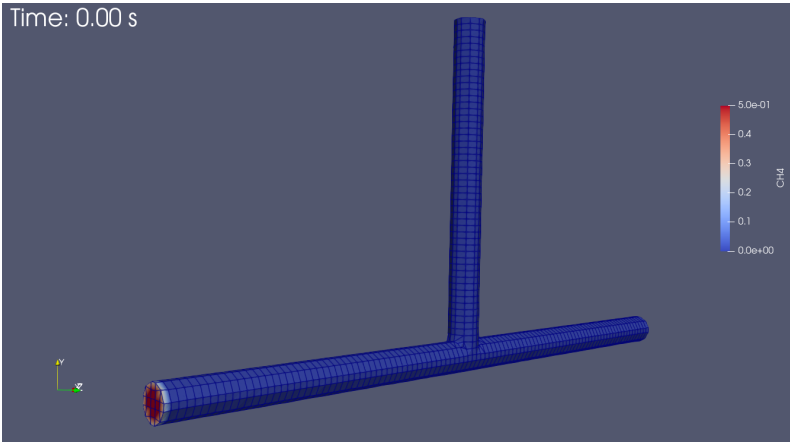


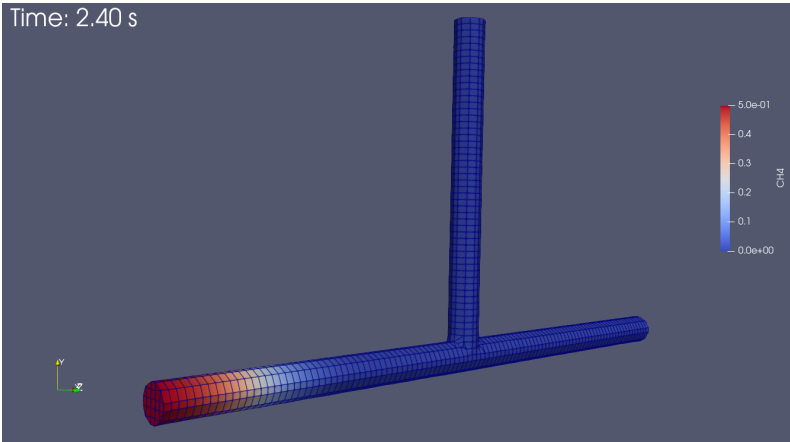
Excerpt from the *snappyHexMeshDict* file.

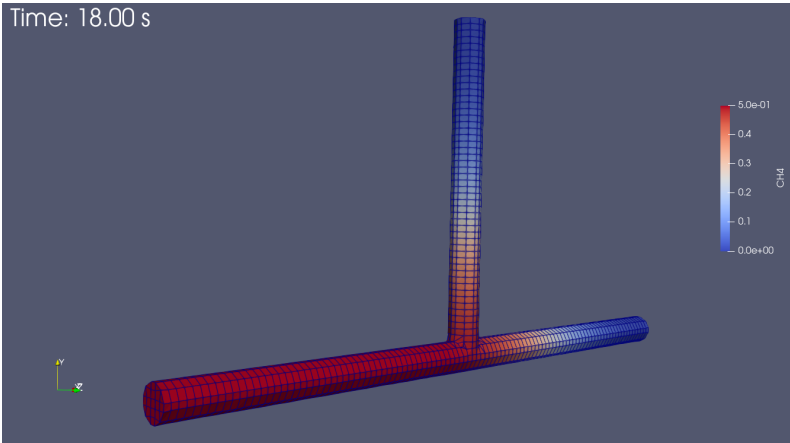
```
1 geometry
2 {
3     tJunction
4     {
5         type triSurfaceMesh;
6         file "tJunction.stl";
7         regions
8         {
9             inlet          {name inlet;}
10            outlet1        {name outlet1;}
11            outlet2        {name outlet2;}
12            walls           {name walls;}
13        }
14    }
15 };
```

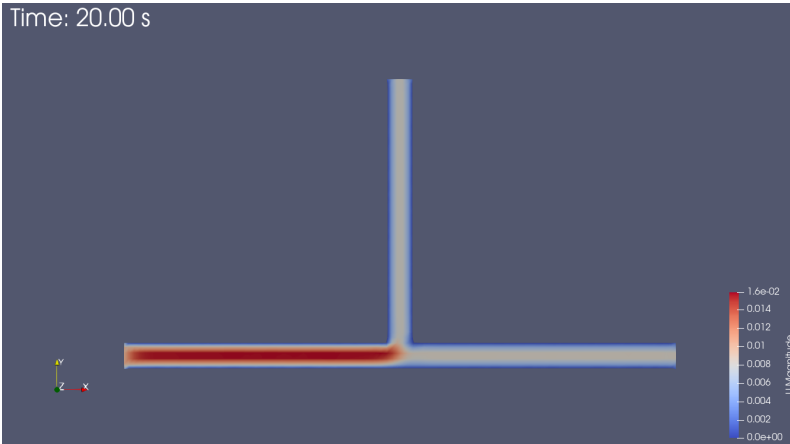




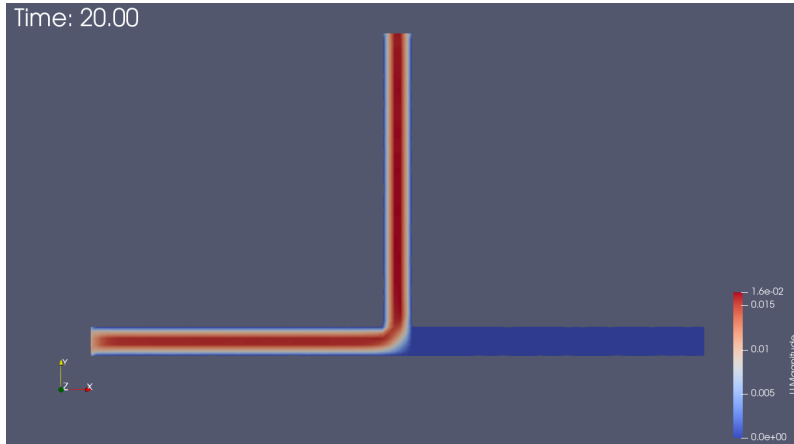




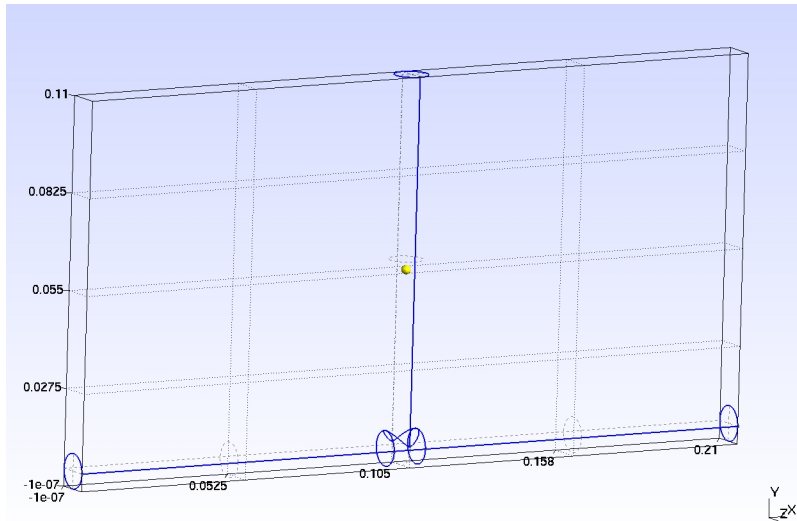




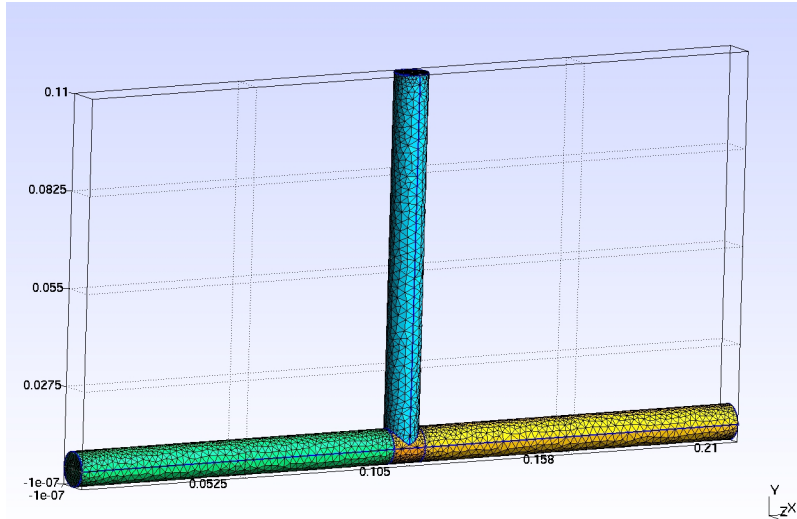
Simulation 05: Gas in a T-Junction with membrane and snappyHexMesh.



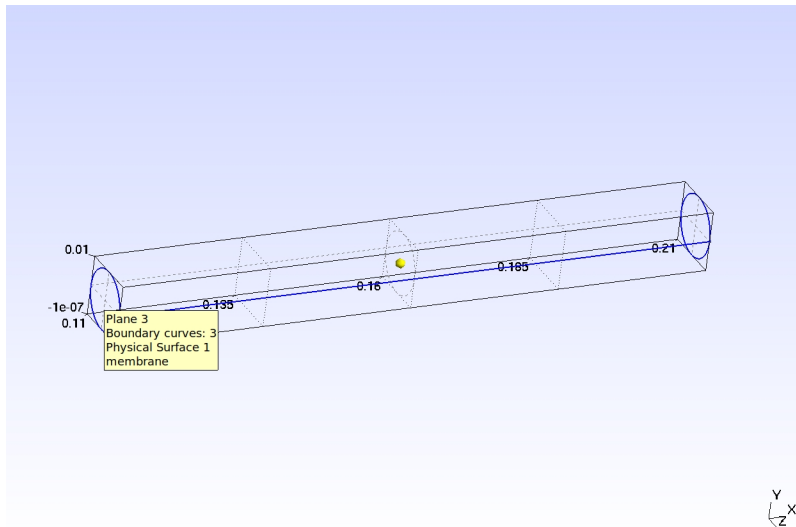
Geometry: T-Junction.



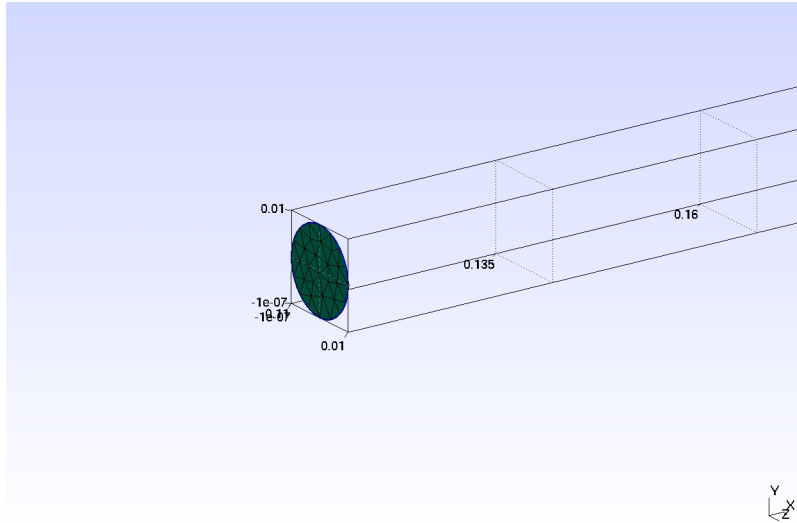
Geometry: T-Junction.



Geometry: membrane. **Note that we need a different stl file for the membrane.**



Geometry: membrane.



We need to place both stl files (T-Junction and membrane) in the *constant/geometry* folder.

We will create the mesh using *snappyHexMesh*, with the background mesh and both *stl* files.

The membrane is added in the *snappyHexMeshDict* file.

Excerpt from the *snappyHexMeshDict* file.

```
1 geometry
2 {
3     tJunction
4     {
5         type triSurfaceMesh;
6         file "tJunction.stl";
7         regions
8         {
9             inlet          {name inlet;}
10             outlet1        {name outlet1;}
11             outlet2        {name outlet2;}
12             walls          {name walls;}
13         }
14     }
15
16     membrane
17     {
18         type triSurfaceMesh;
19         file "membrane.stl";
20     }
21 };
```

Excerpt from the *snappyHexMeshDict* file.

```
1
2  ...
3
4  membrane
5  {
6      level      (0 0);
7      cellZone    permeate;
8      faceZone    membrane;
9      mode        insidePoint;
10     insidePoint (0.15 0.005 0.005);
11 }
12 }
```


We generate the mesh with the command *snappyHexMesh*.

Then we include the membrane with the command *createBaffles*.

We also need to execute the command *setFields*, to set the initial condition for the simulation.

Excerpt from the *setFieldsDict* file.

```
1 defaultFieldValues
2 (
3     //volScalarFieldValue CH4 0.0
4     //volScalarFieldValue CO2 0.0
5     //volScalarFieldValue N2 1.0
6 );
7 regions
8 (
9     zoneToCell
10    {
11        zone permeate;
12        fieldValues
13        (
14            //volScalarFieldValue CH4 1e-4
15            volScalarFieldValue CO2 1.e-10
16            //volScalarFieldValue N2 1.0
17        );
18    }
19 );
```

Copy all field files from the 0 folder to the latest time folder, then execute the command:

```
$ setFields -latestTime
```

The option `-latestTime` tells OpenFOAM to apply the command to the folder with the highest time value.

Important Note: the value of C_{O2} in the permeate zone should be zero. However, we set it to $1e-10$.

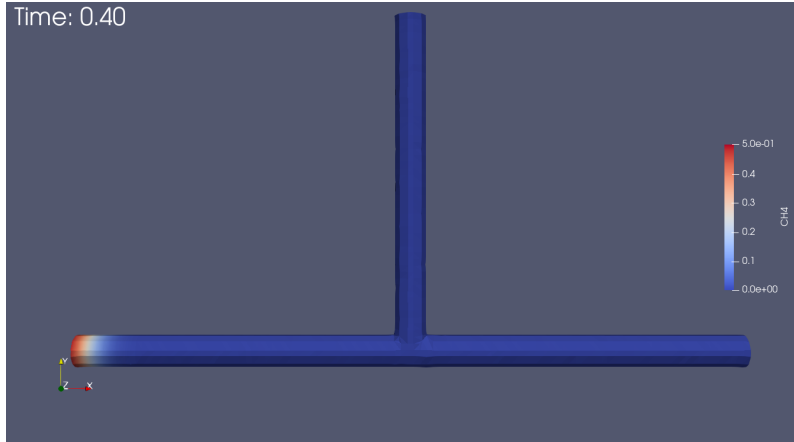
If the initial value is exactly zero, the simulation fails with a memory error. Although the exact cause is unclear, it is likely due to a division by zero when the concentration is zero throughout the region at the start of the simulation.

Using $1e-10$ avoids the error and does not affect the results.

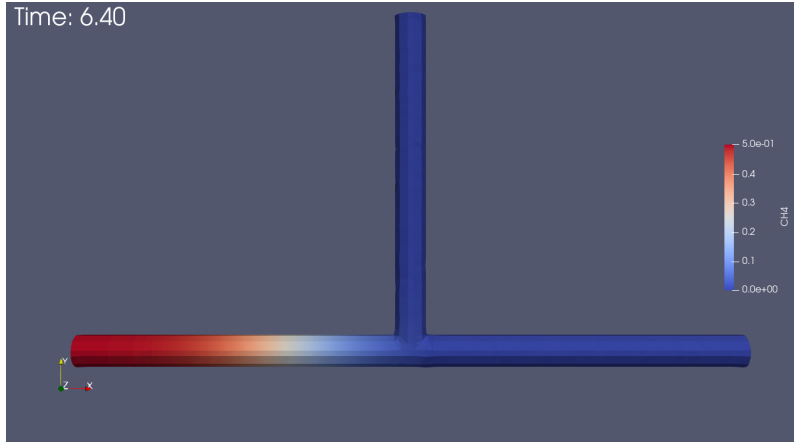
In summary, these are the steps to run the simulation (*scriptRun.py*):

- 1 Generate the geometry using Gmsh (.stl files for the T-Junction and the membrane). Place the .stl files in the constant/geometry folder.
- 2 Generate the background mesh using the command `blockMesh`.
- 3 Generate the final mesh using the command `snappyHexMesh`.
- 4 Create the membrane using the command `createBaffles`.
- 5 Copy the field files (CH₄, CO₂, N₂, p, T, U) from the 0 folder to the folder with the highest time value.
- 6 Define the initial condition using the command `setFields`.
- 7 Run the simulation using the command `foamRun`.

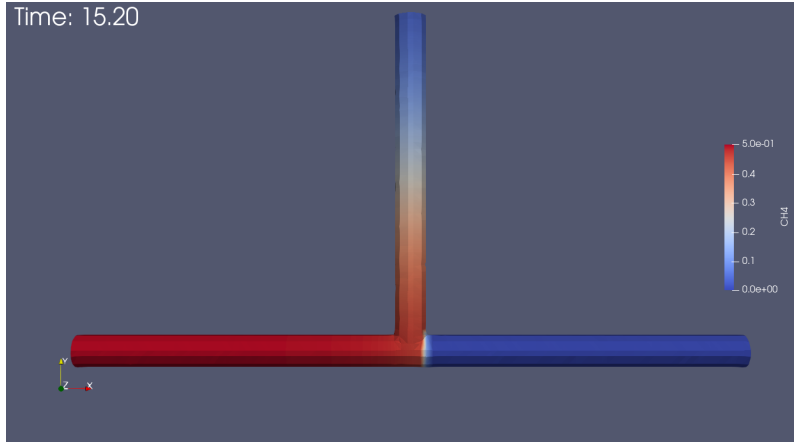
CH₄ mass fraction.



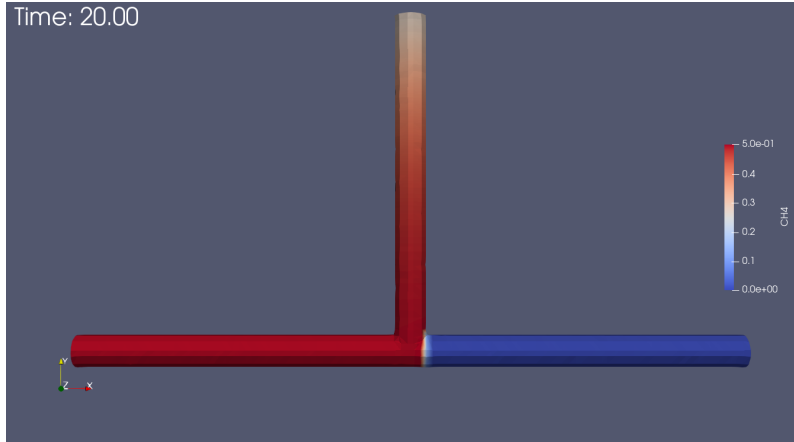
CH₄ mass fraction.



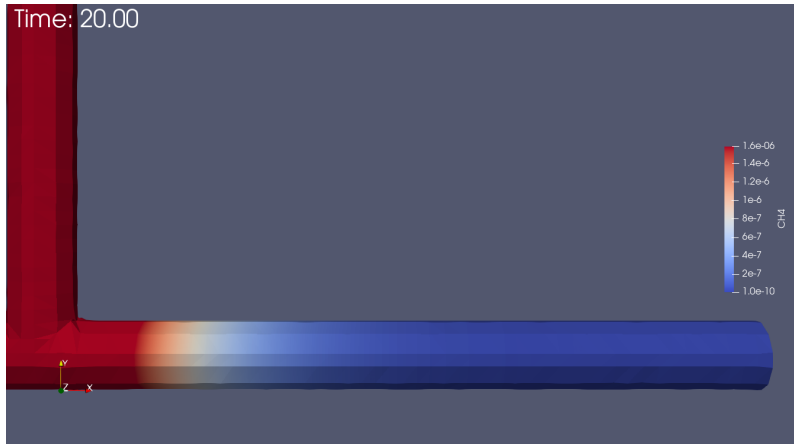
CH₄ mass fraction.



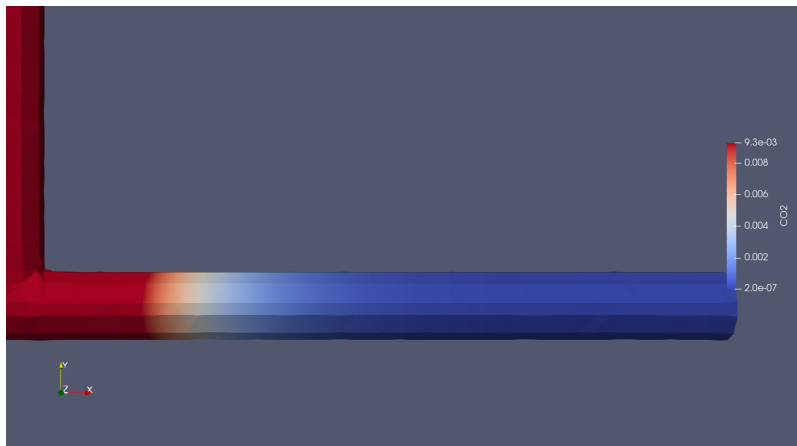
CH₄ mass fraction.

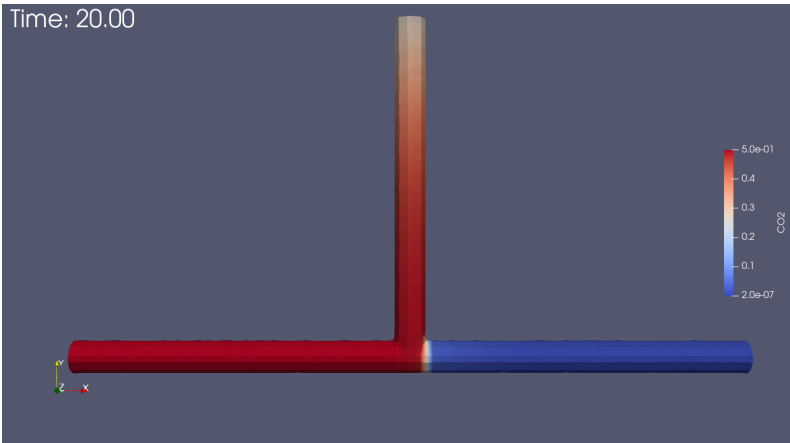


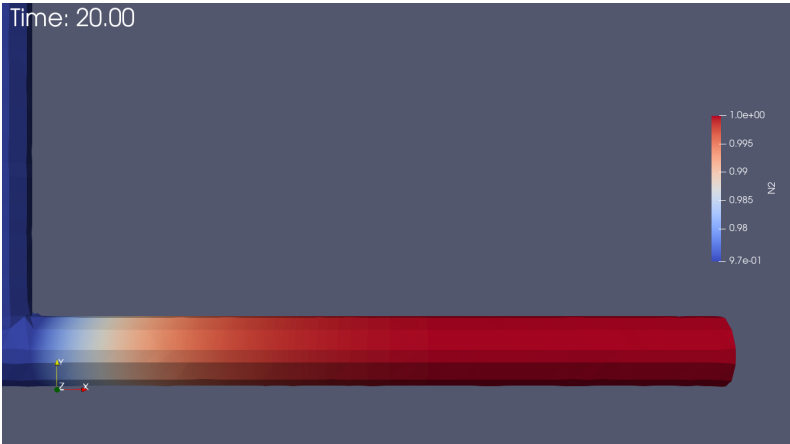
Detail of the CH₄ that has permeated the membrane. Note that some CH₄ crosses the membrane.

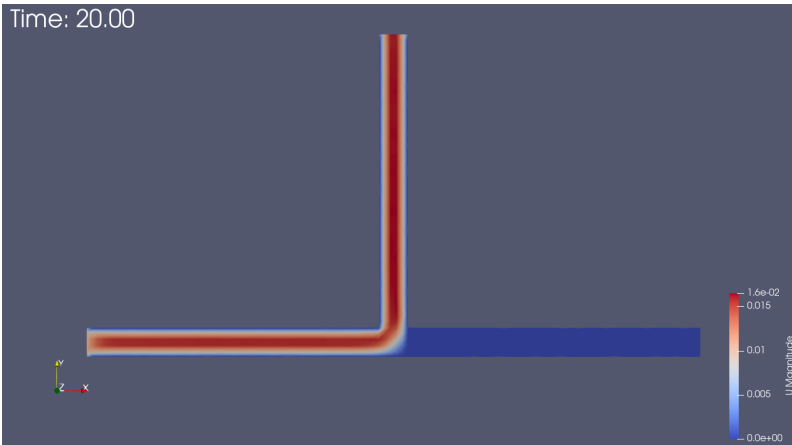


Detail of the CO₂ that has permeated the membrane. Note that more CO₂ crosses the membrane, compared to CH₄.









Velocity magnitude in the permeate side.



Problem 1. Investigate the effect of c_{CO_2} on the outlet concentration of CH₄ using Simulation 05.

Keep $c_{N_2} = c_{CH_4} = 10^{-8} \text{ kg}/(\text{m}^2 \cdot \text{s} \cdot \text{kg})$ and vary c_{CO_2} as follows:
 $10^{-8}, 10^{-7}, 10^{-6}, 10^{-5}, 10^{-4}, 10^{-3}, 10^{-2}, 10^{-1} \text{ kg}/(\text{m}^2 \cdot \text{s} \cdot \text{kg})$.

Plot c_{CO_2} vs. CH₄ concentration at the outlet. Use the patchAverage function to compute CH₄ at the outlet. Ensure the simulation has reached steady state.

Problem 2. Investigate the effect of inlet velocity (U_{inlet}) on CH₄ concentration at the outlet using Simulation 05.

Keep $c_{N_2} = c_{CH_4} = 10^{-8} \text{ kg}/(\text{m}^2 \cdot \text{s} \cdot \text{kg})$ and $c_{CO_2} = 10^{-3} \text{ kg}/(\text{m}^2 \cdot \text{s} \cdot \text{kg})$.

Plot U_{inlet} vs. CH₄ concentration at the outlet.

Sumário

- 1 Gas Flow Simulation
- 2 Baffles (Membranes)
- 3 Simulation: T-Junction with Membrane
- 4 SnappyHexMesh
- 5 Mesh Generation with SHM
- 6 Simulation: 3D T-Junction with Membrane and SHM
- 7 Conclusion

Key topics from **Lecture 02**.

- Multicomponent gas simulation
- Membranes as Baffles
- snappyHexMesh

Now we have everything that we need to run the complete module.

Next Lecture:

- Run the complete simulation of the membrane gas separation module